

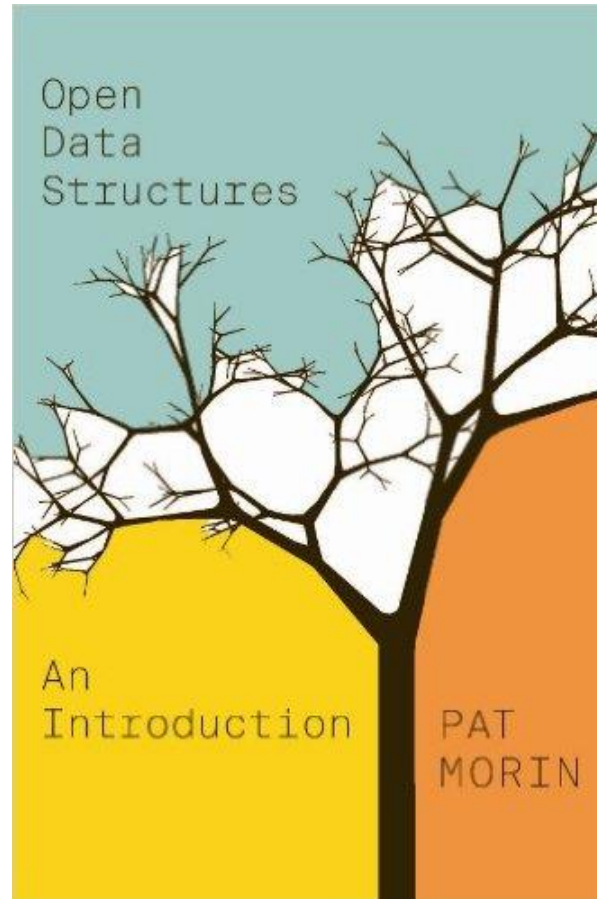
Graph Traversals

COMP 2402
Abstract Data Types & Algorithms

Readings

Today's class

- Graphs
 - Chapter 12.3



Graphs

	Adjacency Matrix	Adjacency List
addEdge	$O(1)$	$O(1)$
removeEdge	$O(1)$	$O(\deg(i))$
hasEdge	$O(1)$	$O(\deg(i))$
outEdges	$O(n)$	$O(1)$
inEdges	$O(n)$	$O(n + m)$
Space used	$O(n^2)$	$O(n + m)$

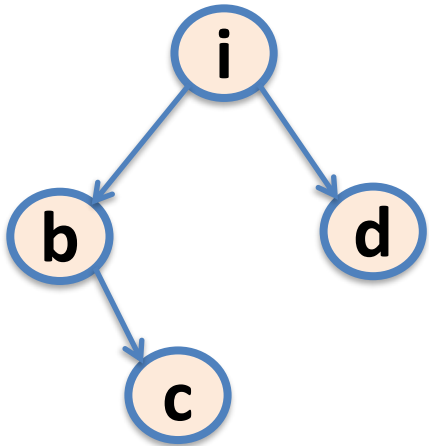
Graph Explorations

We have already seen **breadth-first** and **depth-first search** algorithms for binary trees.

We can use these (slightly modified) to explore graphs: starting with some vertex we find all vertices that are **reachable** from it.

Breadth-First Search

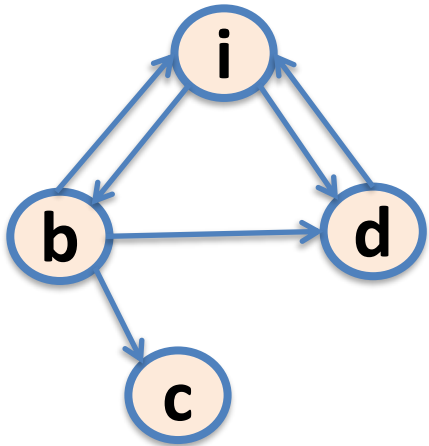
In a **breadth-first search** of a graph we start with a node **i**, and visit the neighbours of **i**, then visit all the neighbours of the neighbours of **i**, etc.



For a binary tree
we used a **Queue**

Breadth-First Search

In a **breadth-first search** of a graph we start with a node *i*, and visit the neighbours of *i*, then visit all the neighbours of the neighbours of *i*, etc.

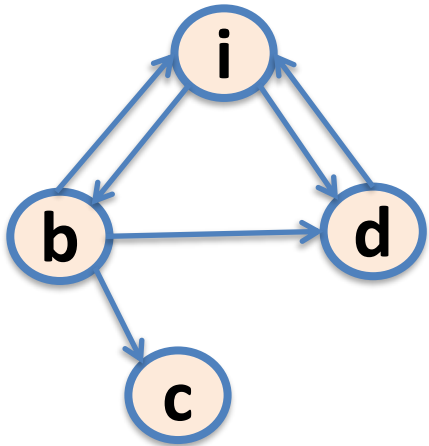


It's a bit more complicated with graphs.

We need to keep track of the nodes we have already discovered

Breadth-First Search

In a **breadth-first search** of a graph we start with a node i , and visit the neighbours of i , then visit all the neighbours of the neighbours of i , etc.



The execution of the BFS constructs a **BFS search tree**.

You ***will*** be asked about this

Breadth-First Search

```
dfs(G, root)
    seen = boolean array of size n
    q = empty queue
    q.add(root)
    seen[root] = true
    while q is not empty do
        i = q.remove()
        for each vertex j in outEdges(i) do
            if seen[j] is false
                q.add(j)
                seen[j] = true
```

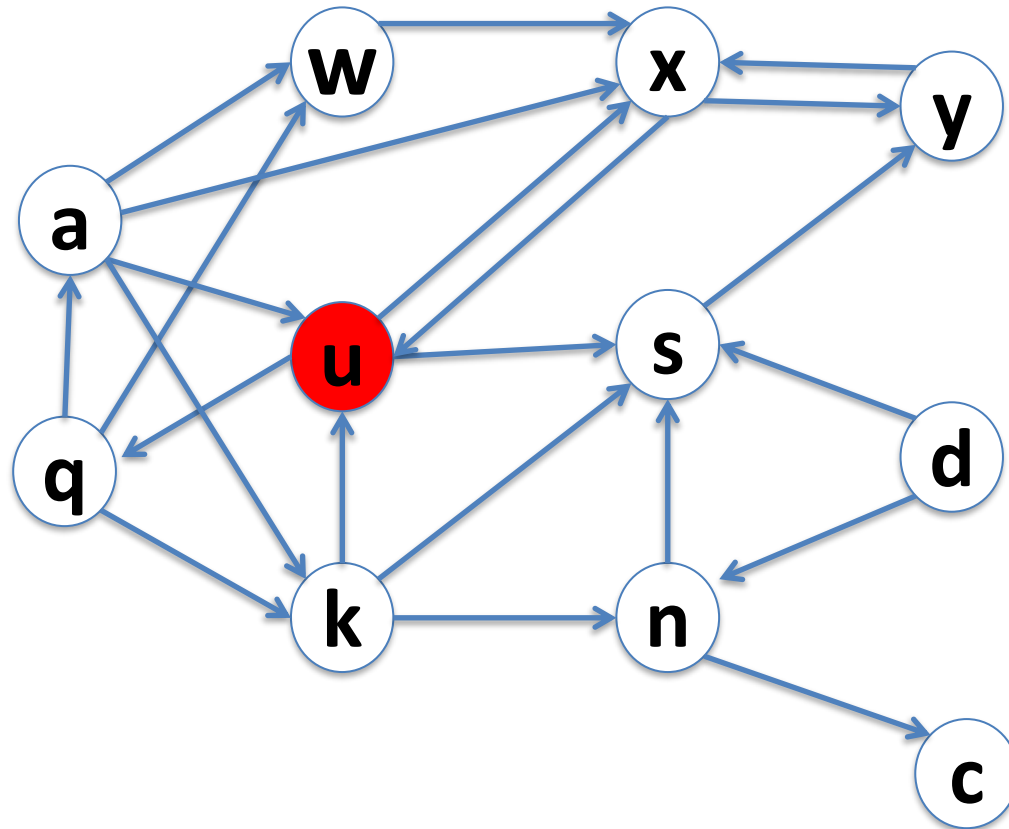

Breadth-First Search

```
bfs(G, root)
  seen = boolean array of size n
  q = empty queue
  q.add(root)
  seen[root] = true
  while q is not empty do
    i = q.remove()
    for each vertex j in outEdges(i) do
      if seen[j] is false
        q.add(j)
        seen[j] = true
```

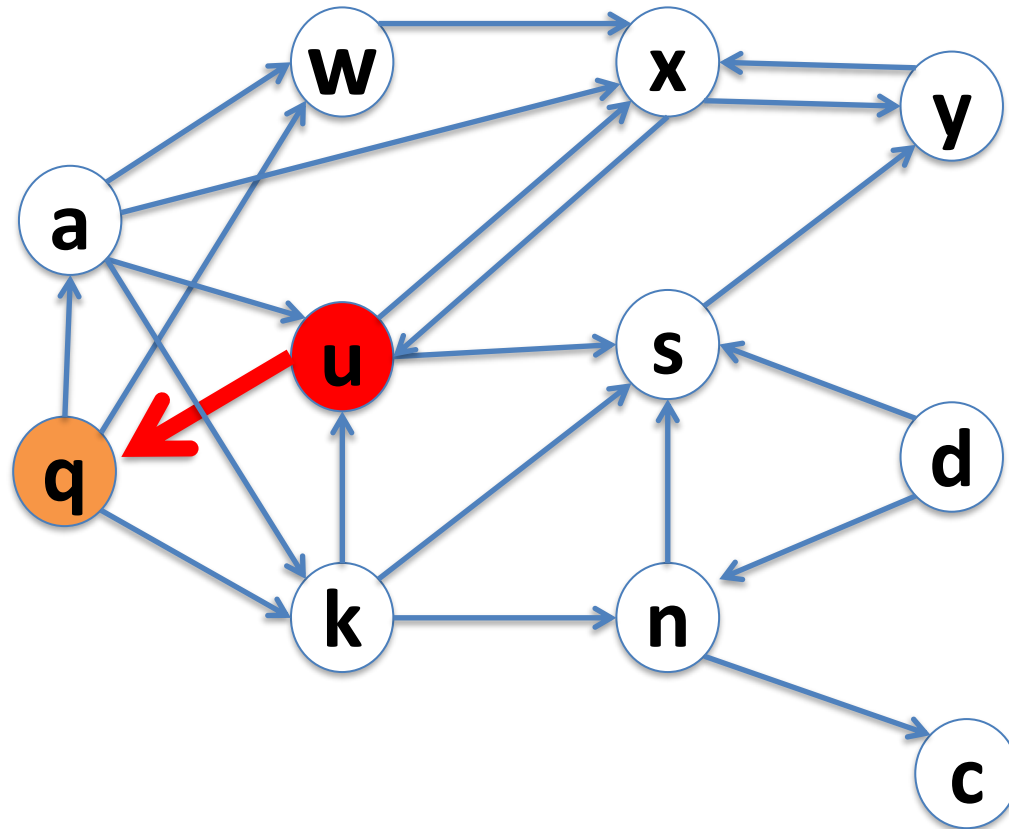


Order these in
increasing order

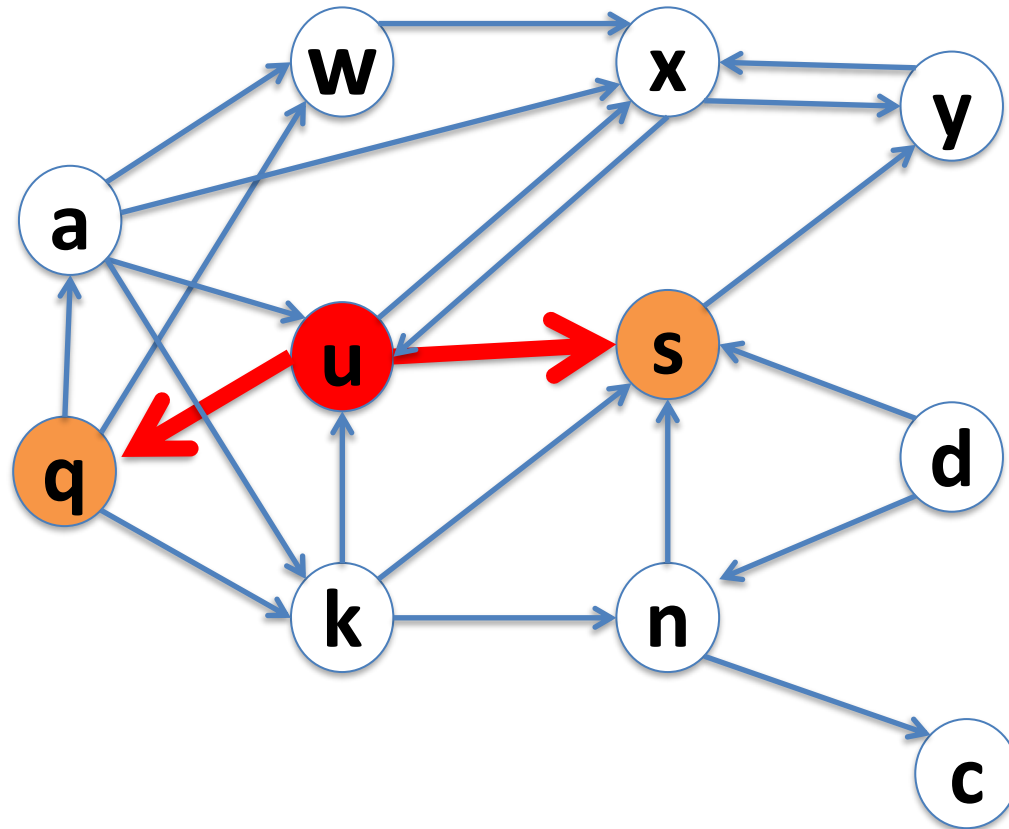
Breadth-First Search



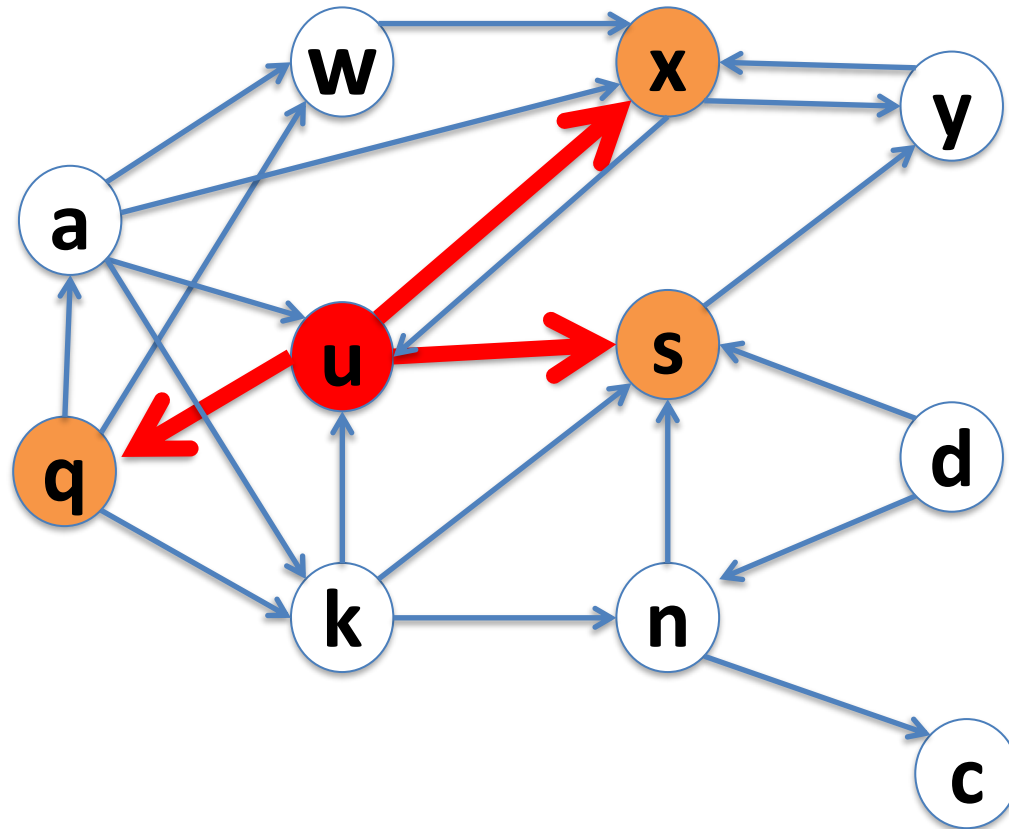
Breadth-First Search



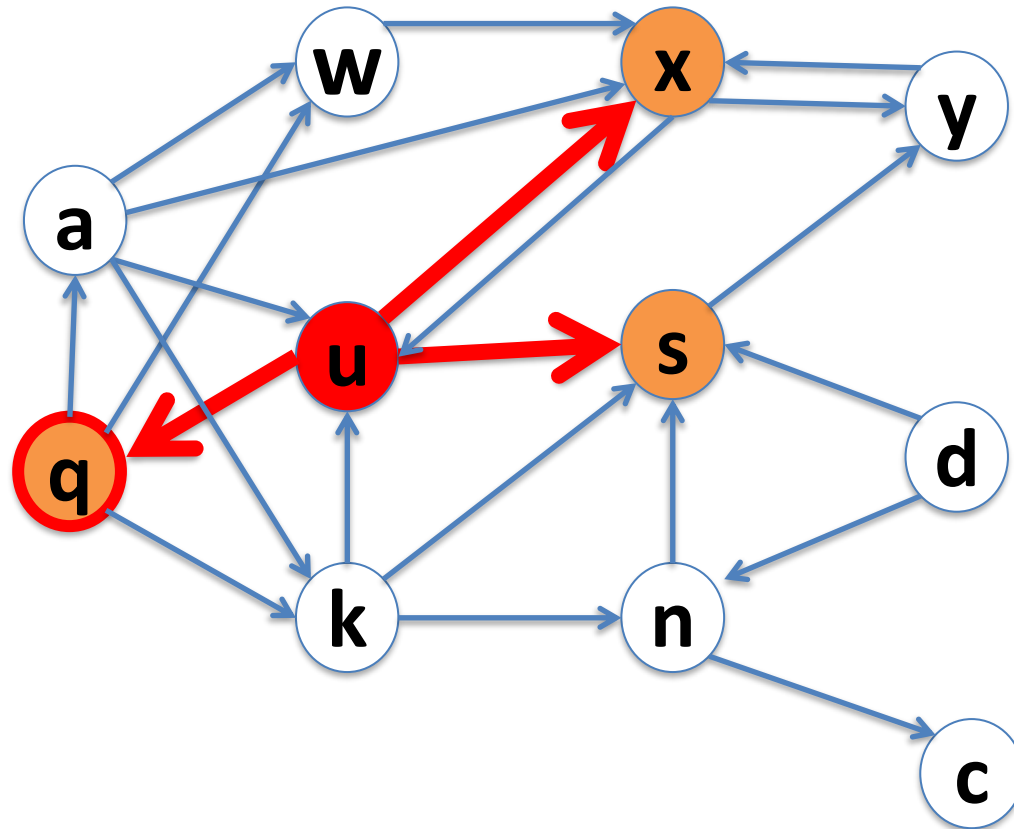
Breadth-First Search



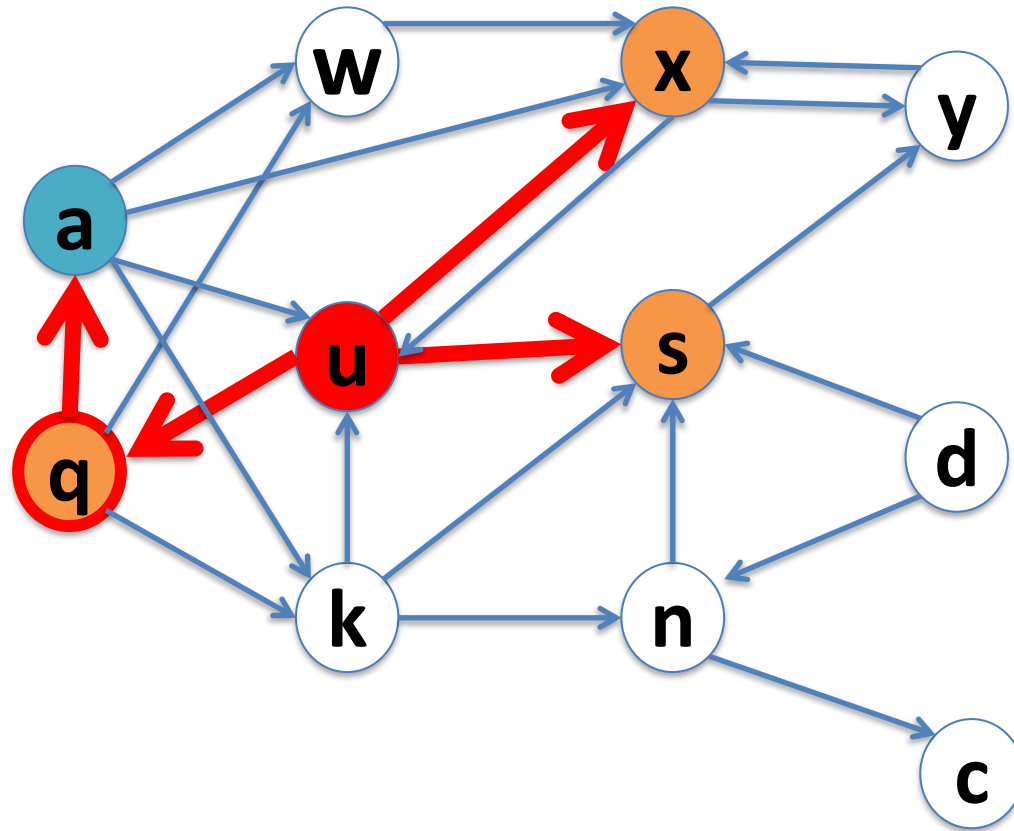
Breadth-First Search



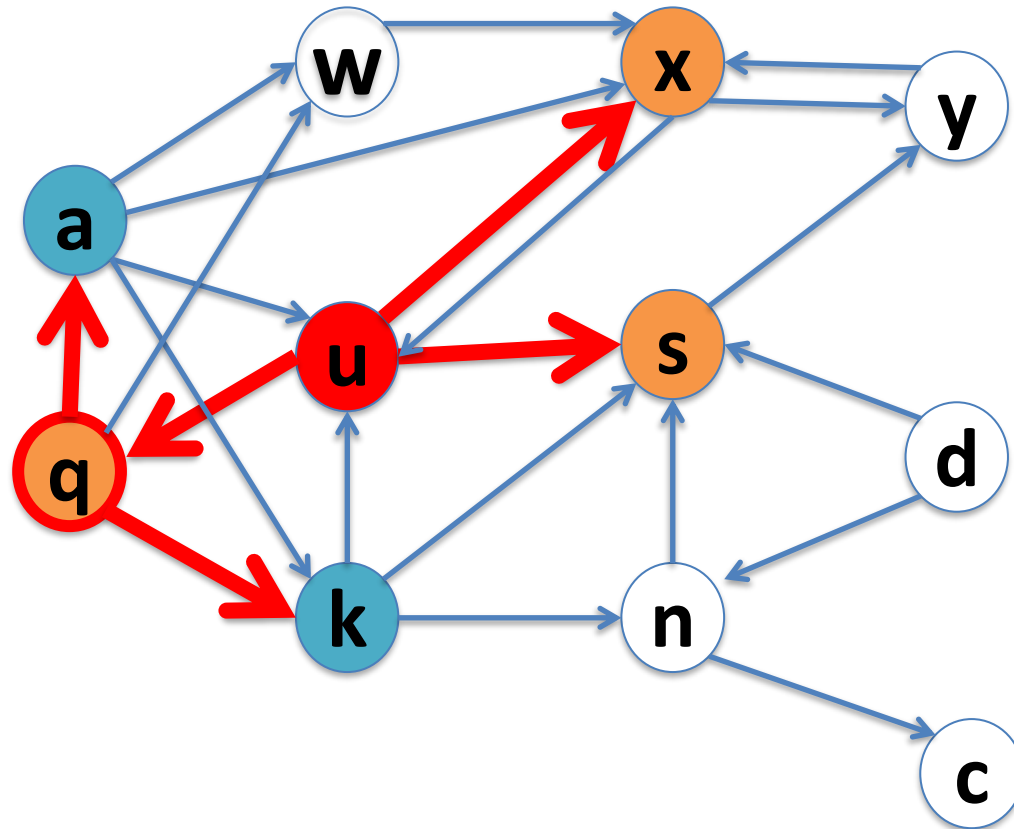
Breadth-First Search



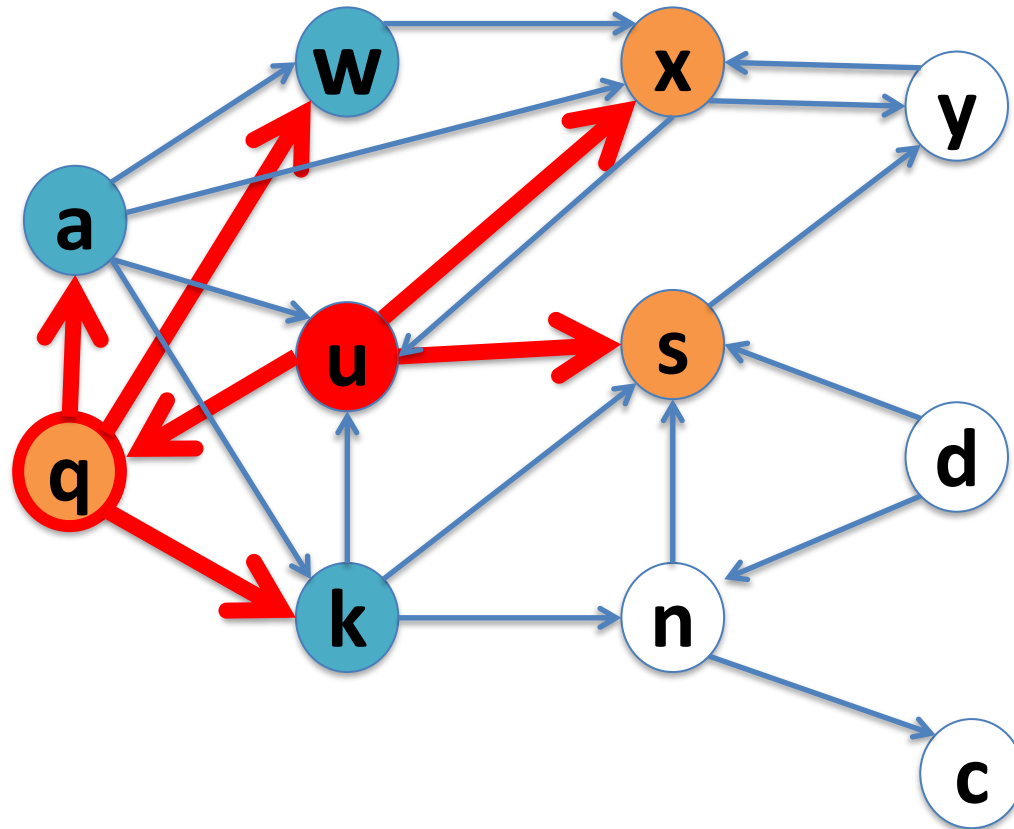
Breadth-First Search



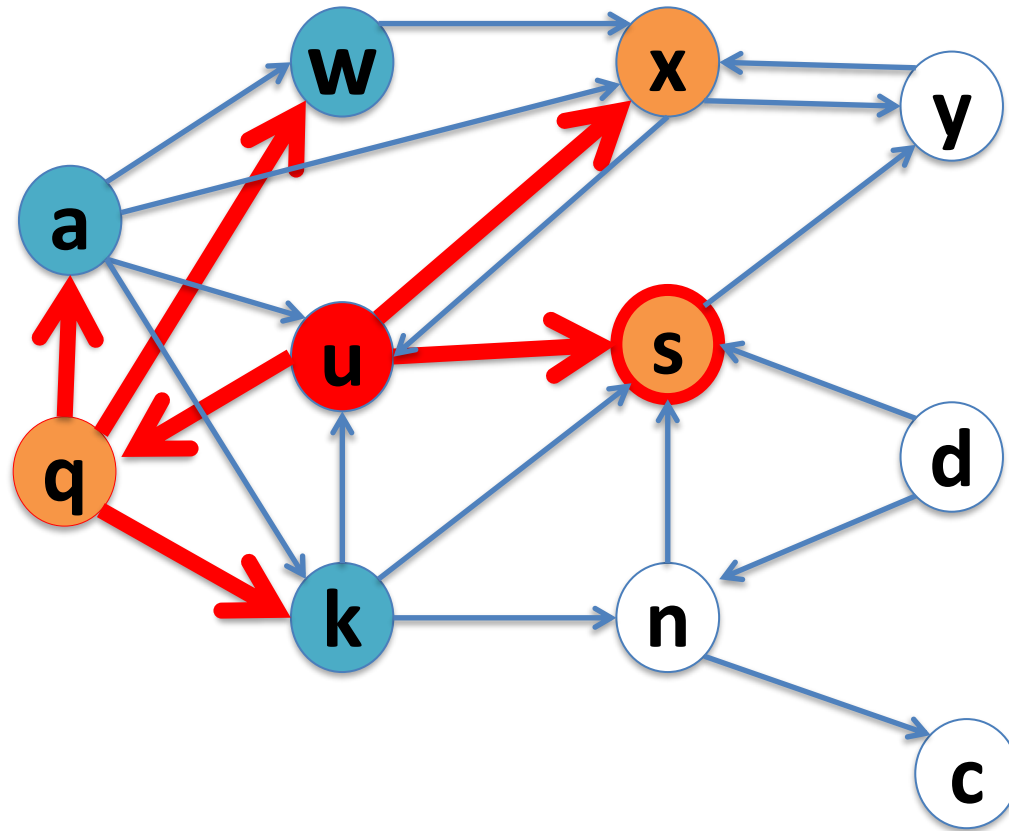
Breadth-First Search



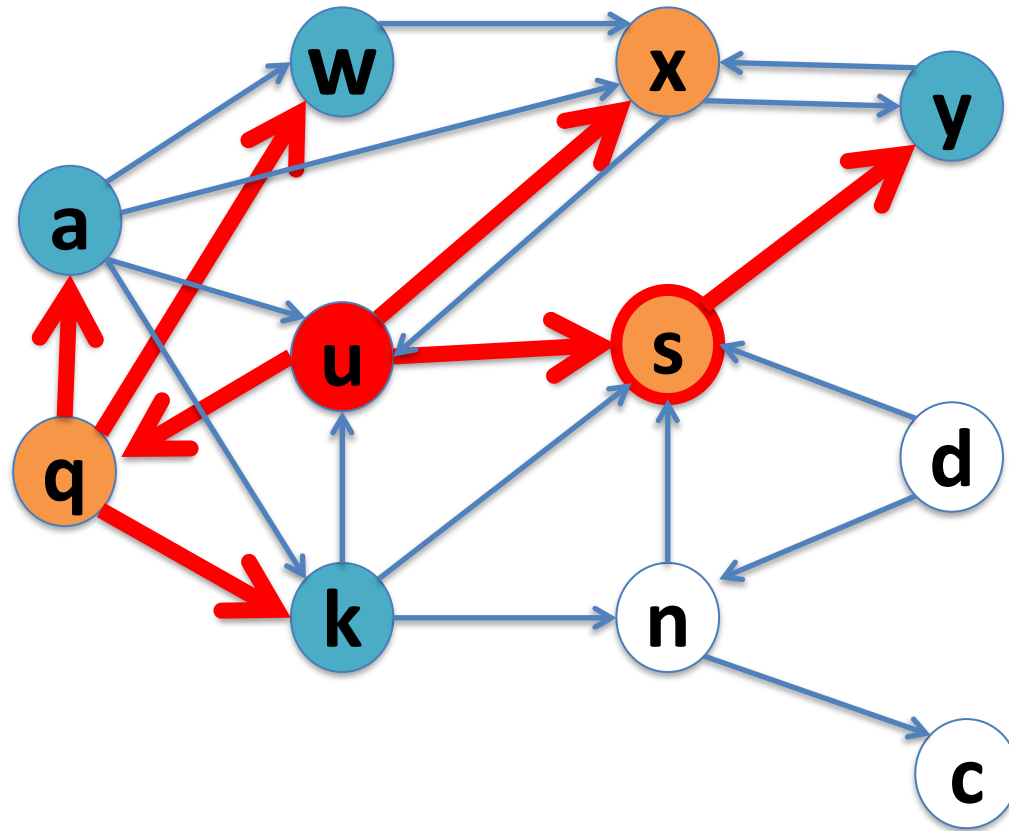
Breadth-First Search



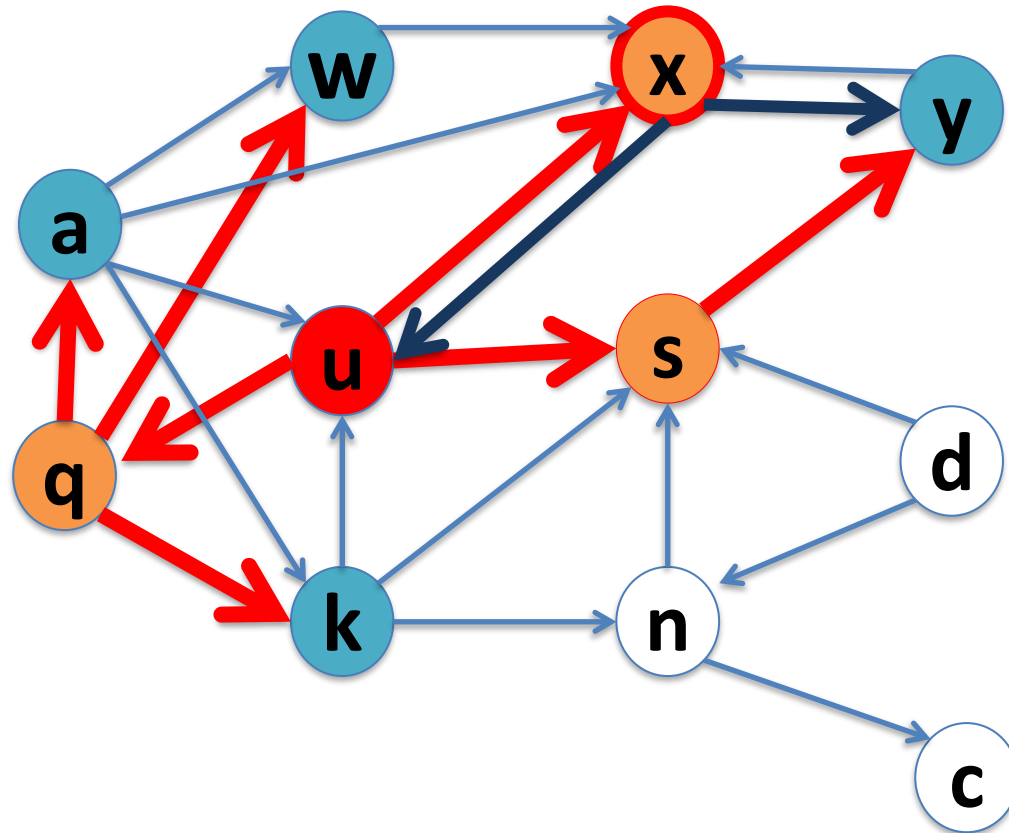
Breadth-First Search



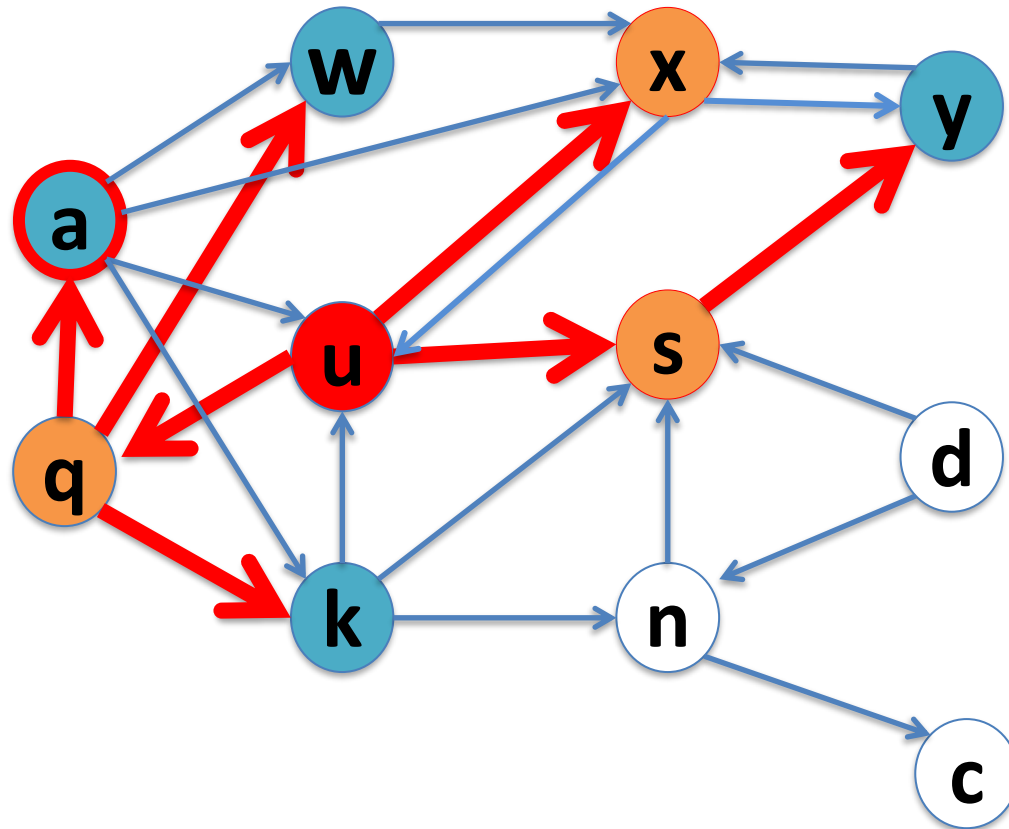
Breadth-First Search



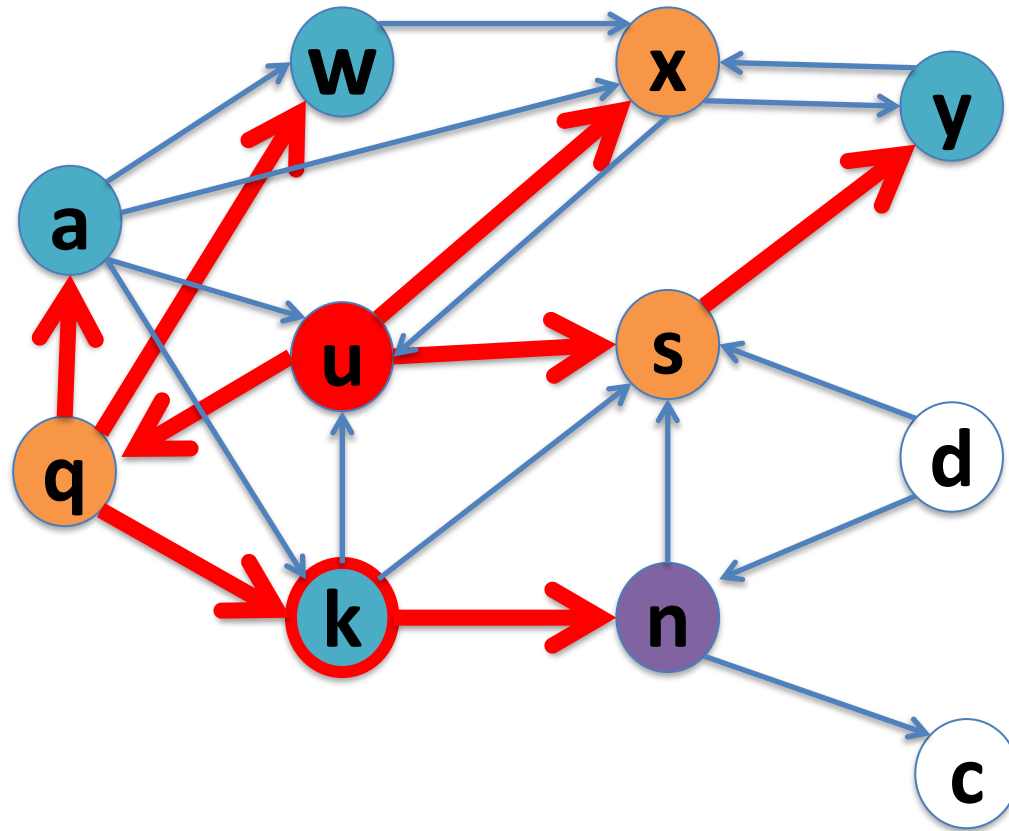
Breadth-First Search



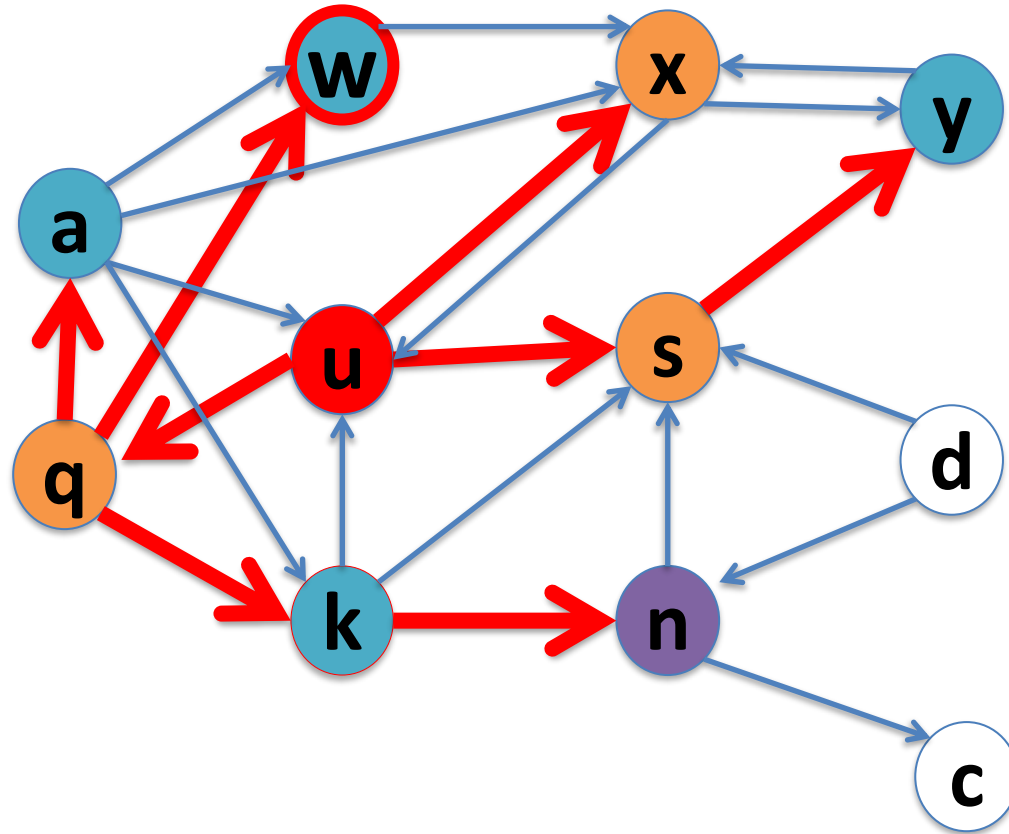
Breadth-First Search



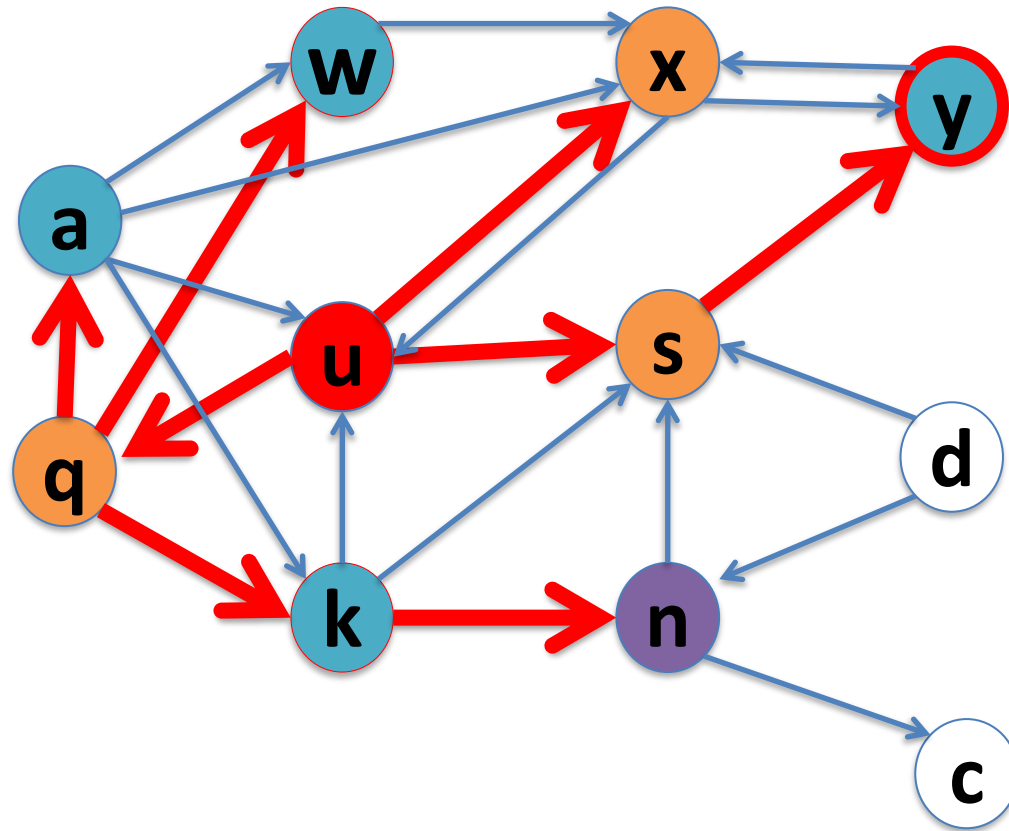
Breadth-First Search



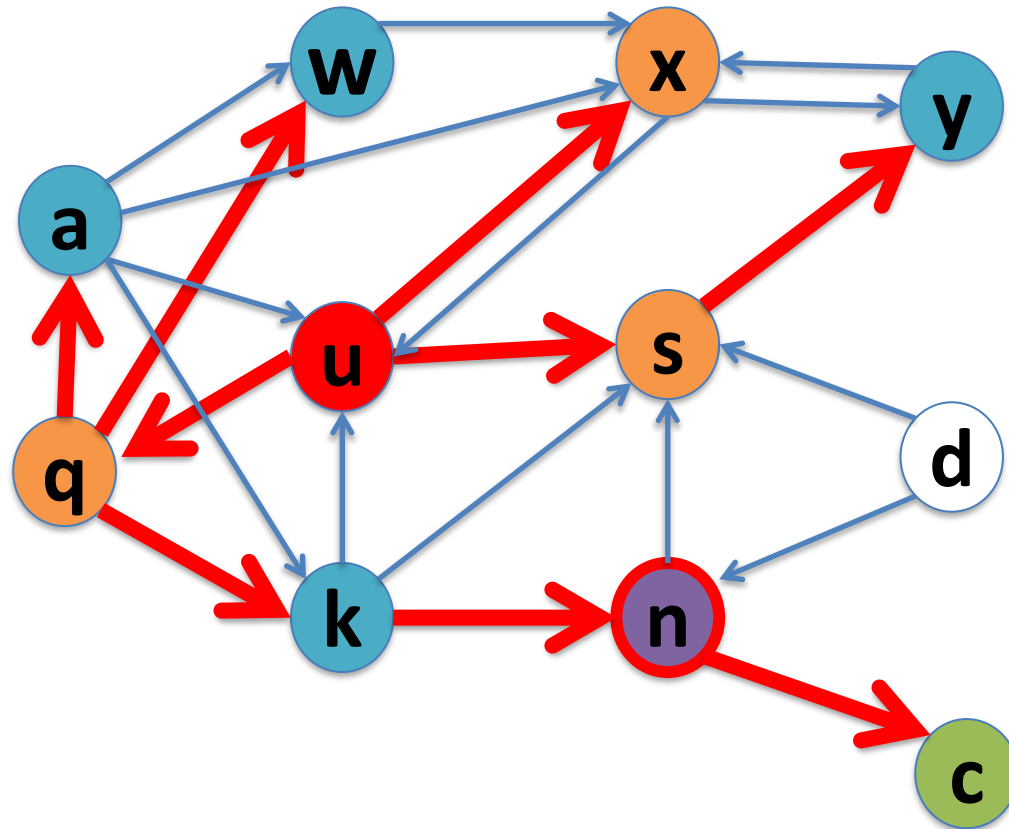
Breadth-First Search



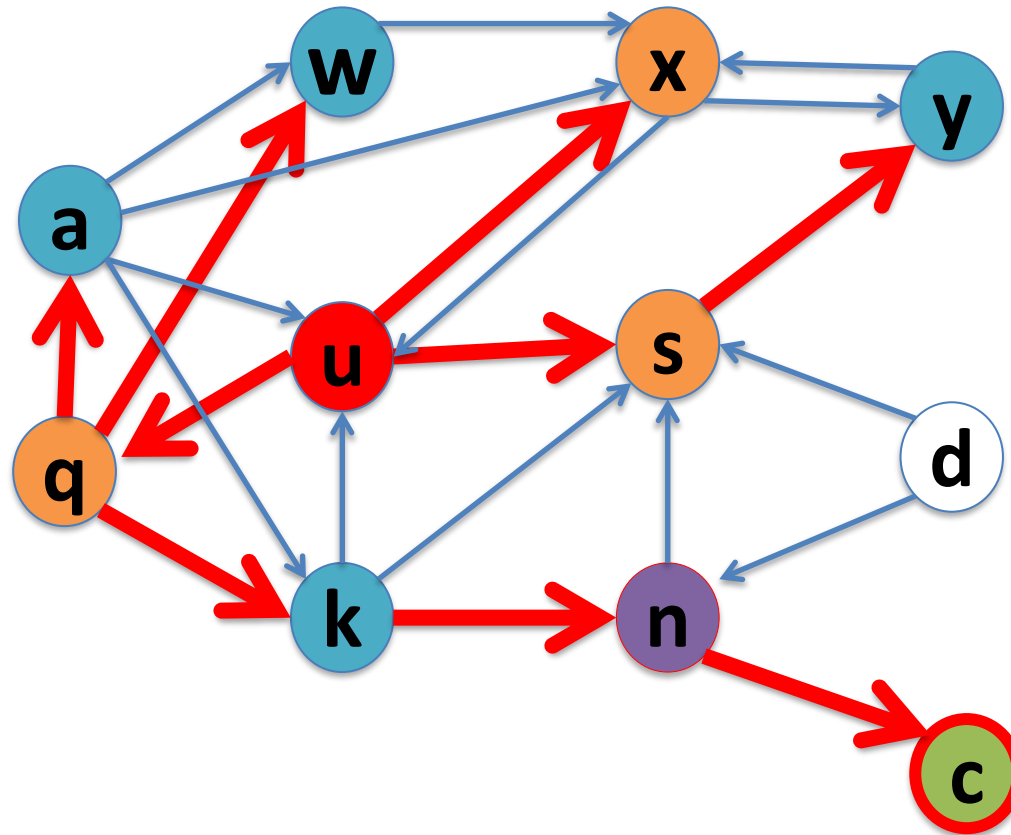
Breadth-First Search



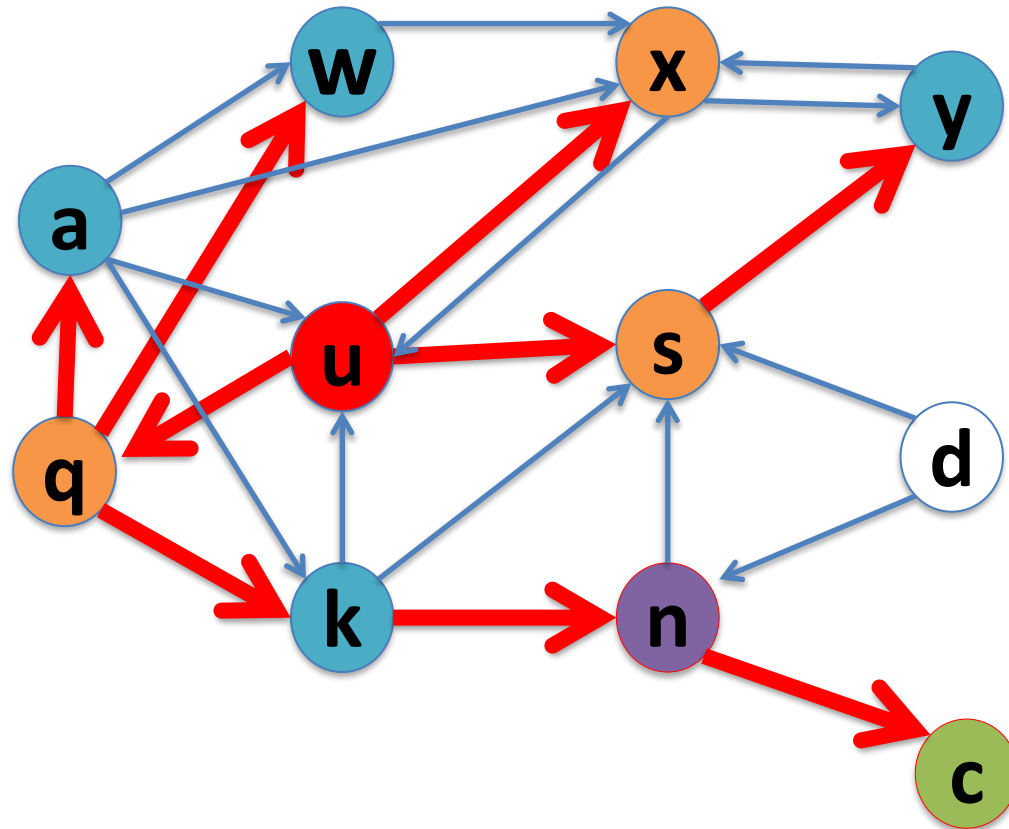
Breadth-First Search



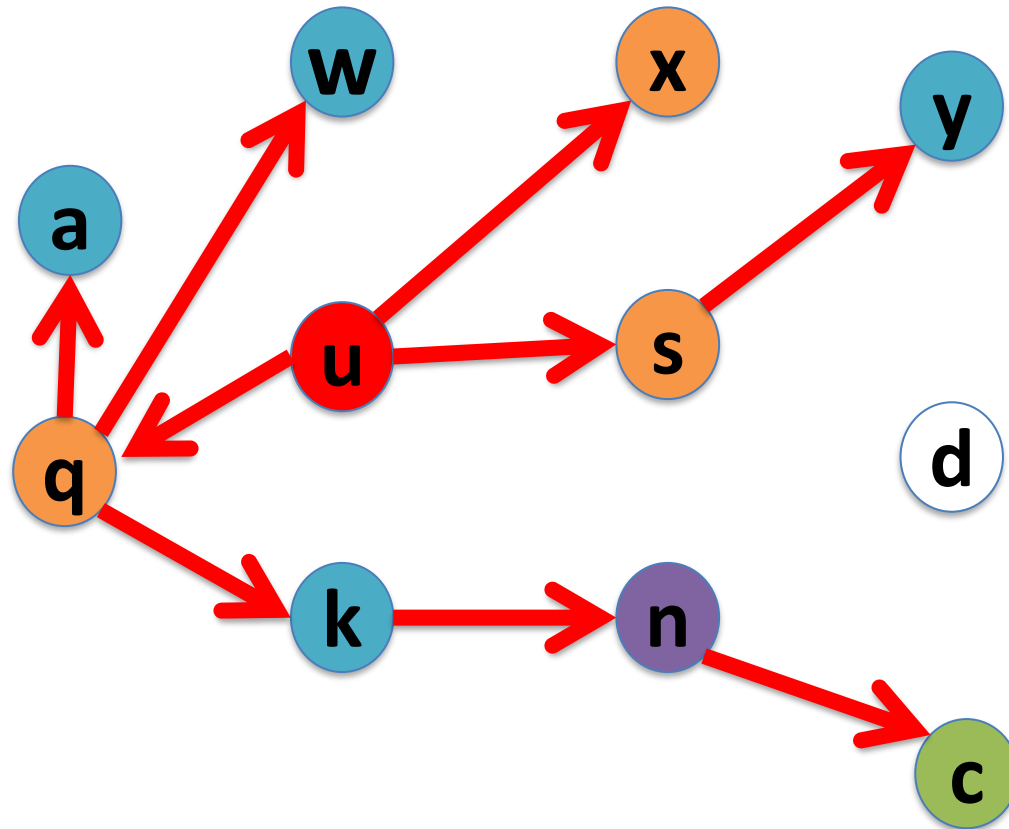
Breadth-First Search



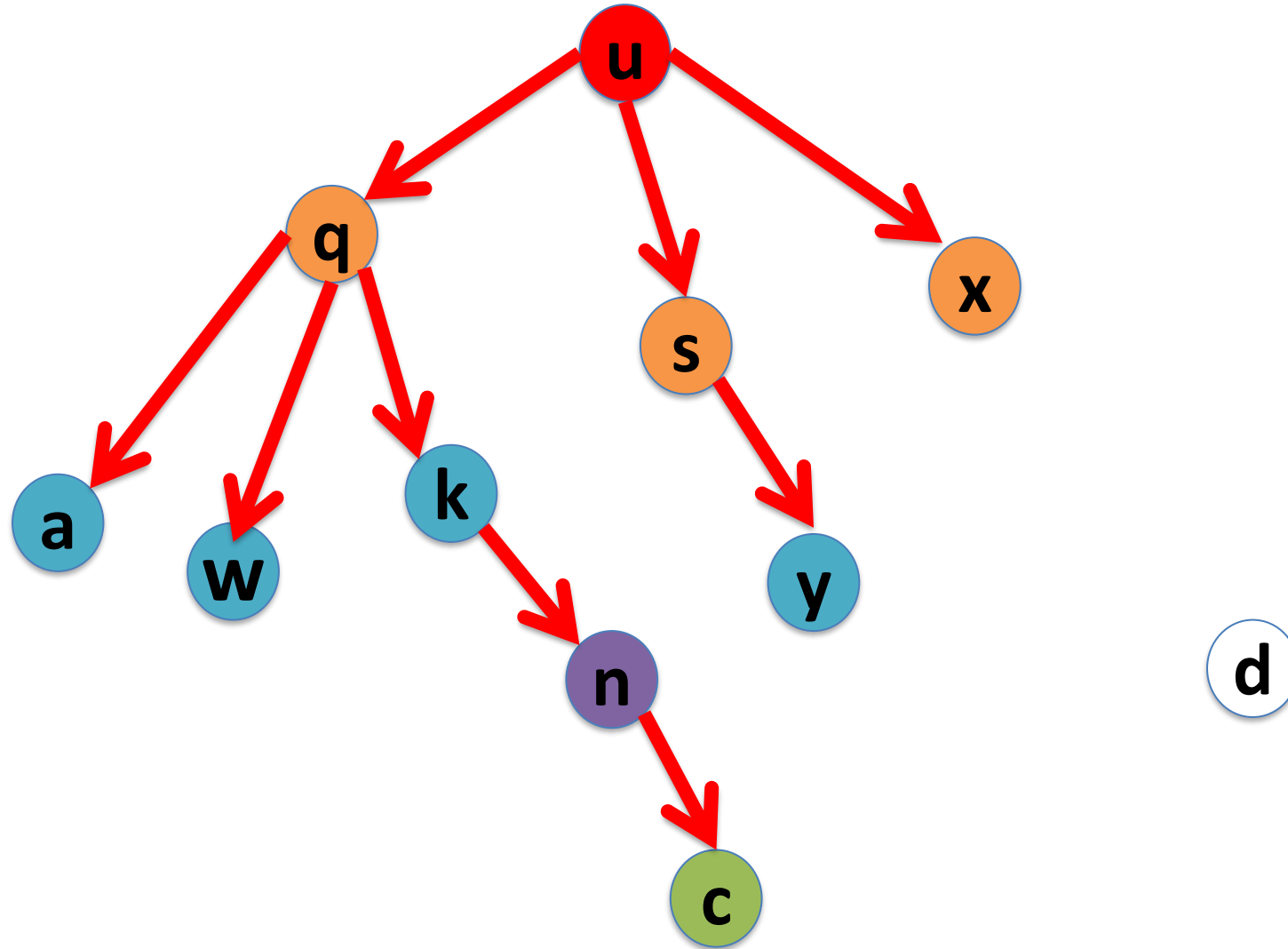
Breadth-First Search



Breadth-First Search



Breadth-First Search



Breadth-First Search

What is the runtime of **breadth-first search**?

- Adjacency Matrix
- Adjacency Lists

Breadth-First Search

```
bfs(G, root)
  seen = boolean array of size n
  q = empty queue
  q.add(root)
  seen[root] = true
  while q is not empty do
    i = q.remove()
    for each vertex j in outEdges(i) do
      if seen[j] is false
        q.add(j)
        seen[j] = true
```



Order these in
increasing order

Breadth-First Search

What is the runtime of **breadth-first search**?

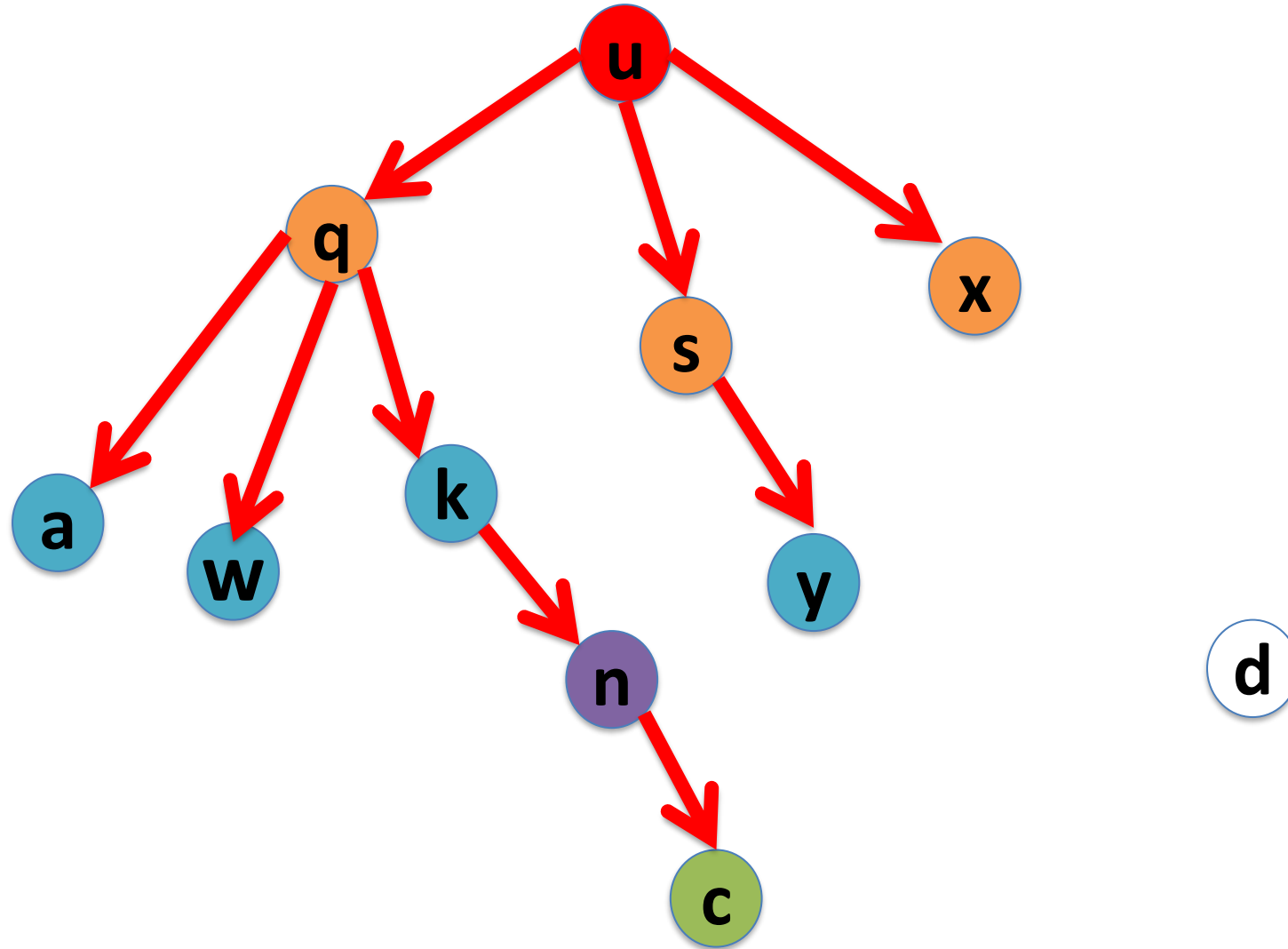
- Adjacency Matrix

$$O(n^2)$$

- Adjacency Lists

$$O(n + m)$$

Breadth-First Search



Breadth-First Search

In a **breadth-first search** of a graph starting with some vertex **u**, we find the **shortest path** from **u** to all other vertices reachable from **u**.

(More about this in COMP3804)

Depth-First Search

In a **depth-first search** we start with some node r and keep taking steps (following edges) while possible. Once we get stuck, we backtrack and try another path; repeating this until we have explored all the graph that is possible.

Like the BFS, we need to keep track of nodes: nodes we haven't seen, nodes we have seen but are not done with and nodes we are done with.

Depth-First Search

In a **depth-first search** we start with some node r and keep taking steps (following edges) while possible. Once we get stuck, we backtrack and try another path; repeating this until we have explored all the graph that is possible.

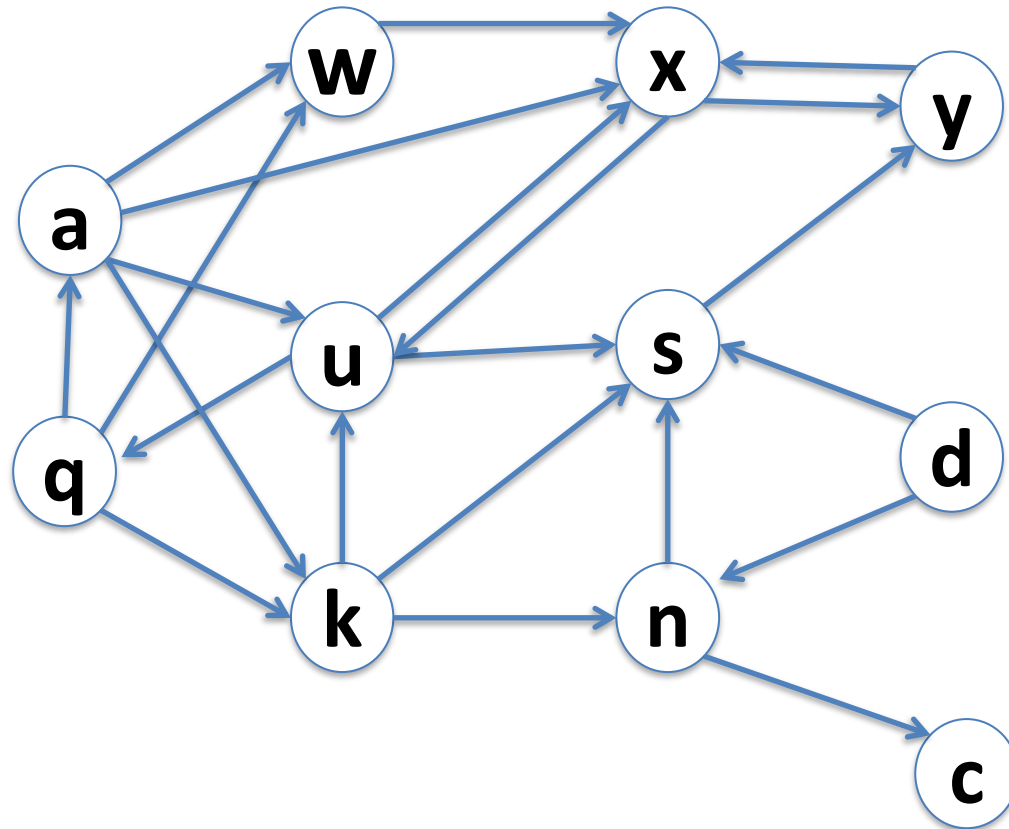
Like the BFS, the DFS algorithm constructs a DFS **search tree**. (When choosing which node to visit always choose the smallest first!)

Depth-First Search

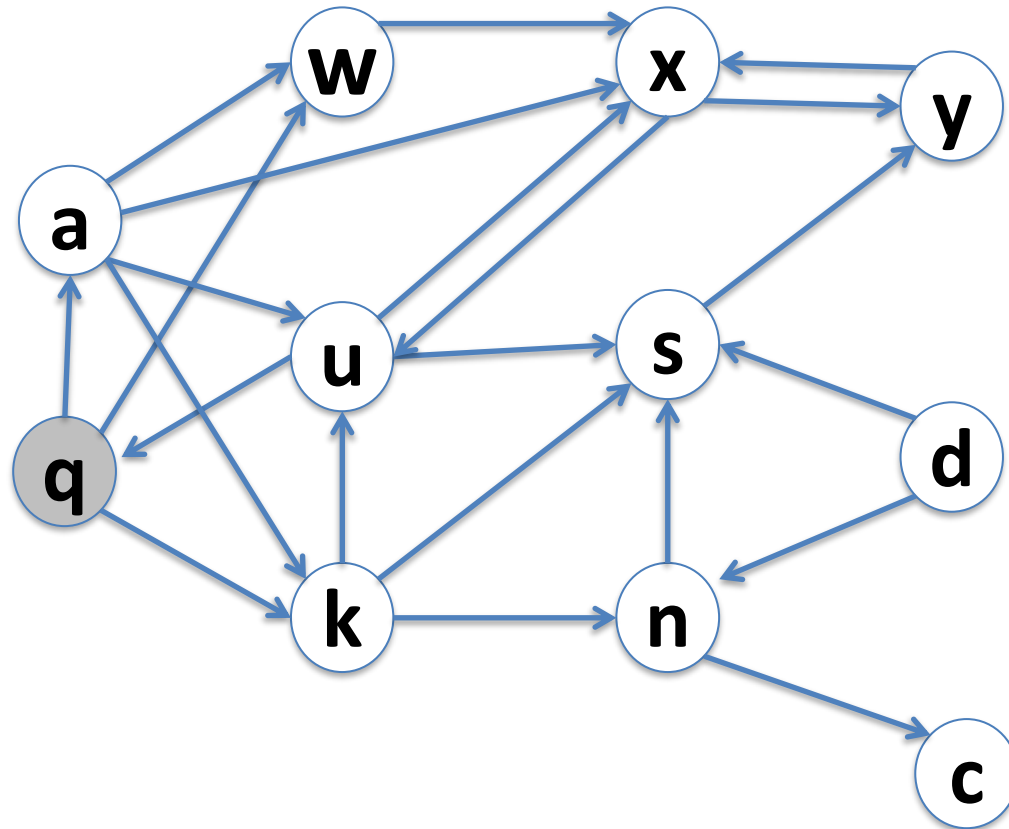
Just like we did for binary trees, we can do a **depth-first search** using a **Stack**.

We'll push elements onto the stack in **DECREASING** order, so that when they get popped of, they are in **INCREASING** order.

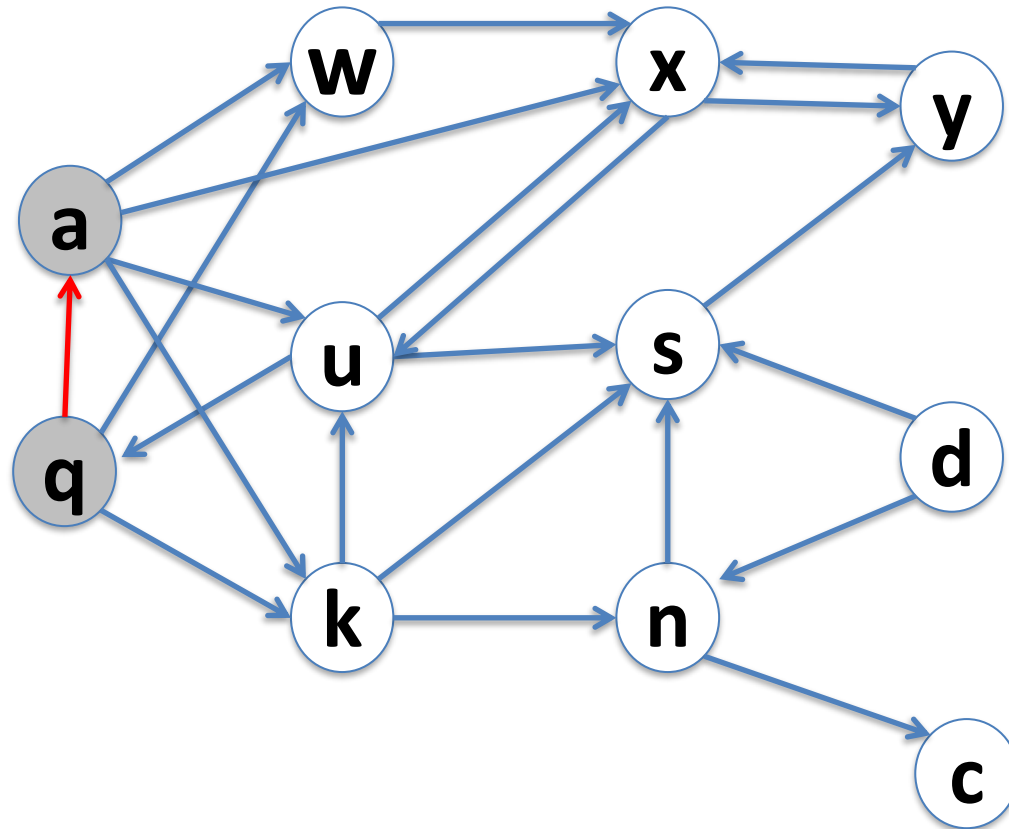
Depth-First Search



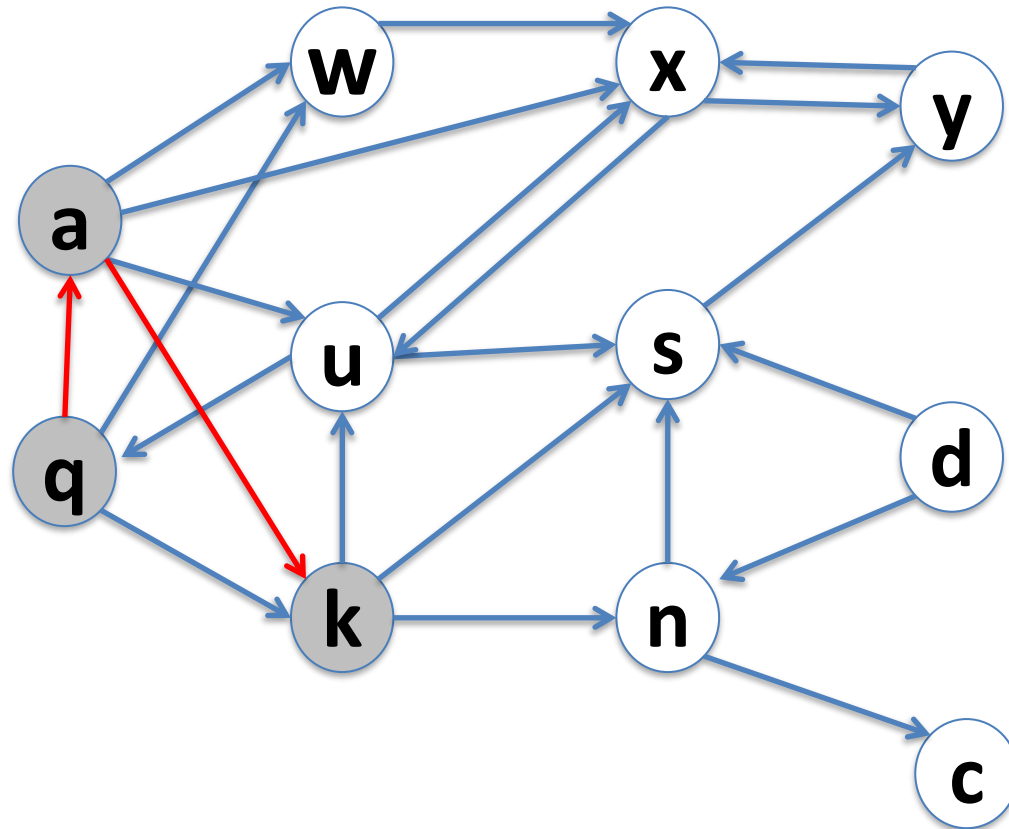
Depth-First Search



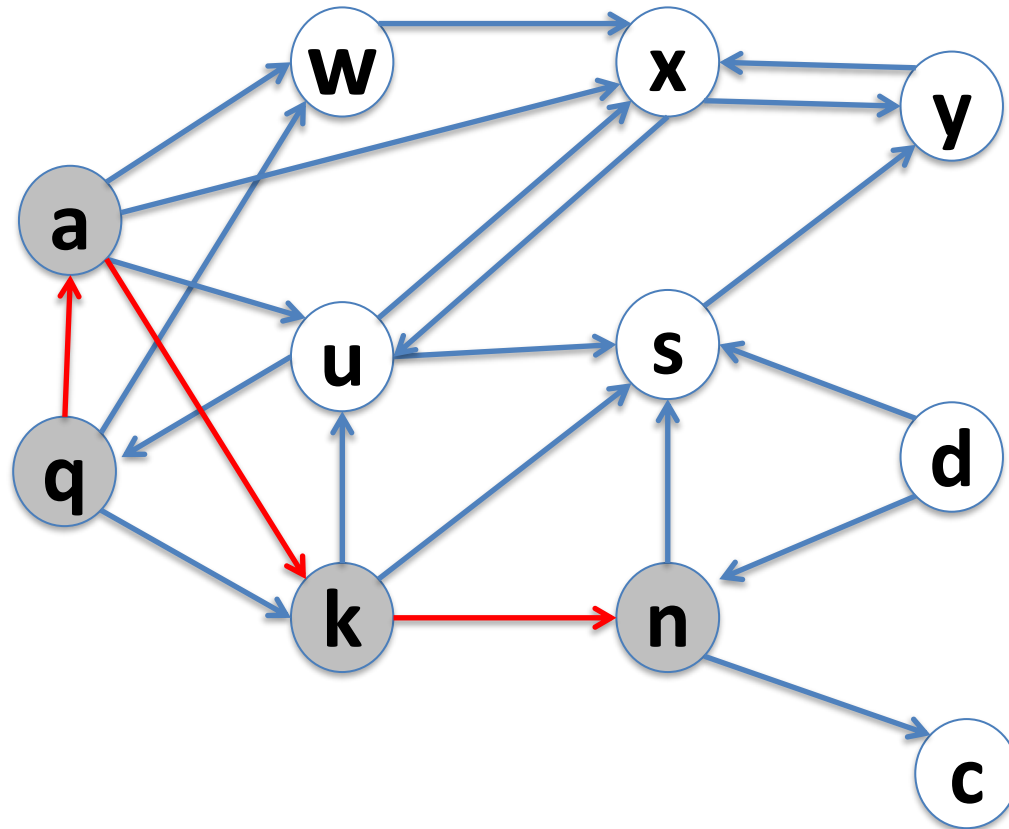
Depth-First Search



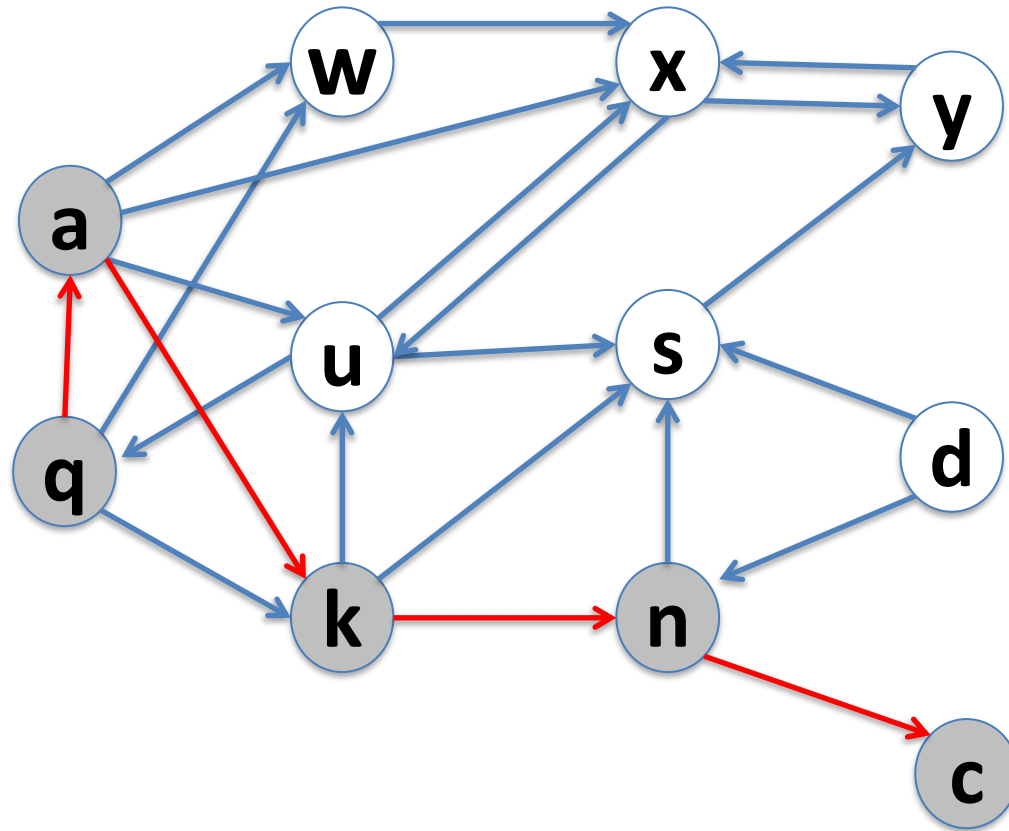
Depth-First Search



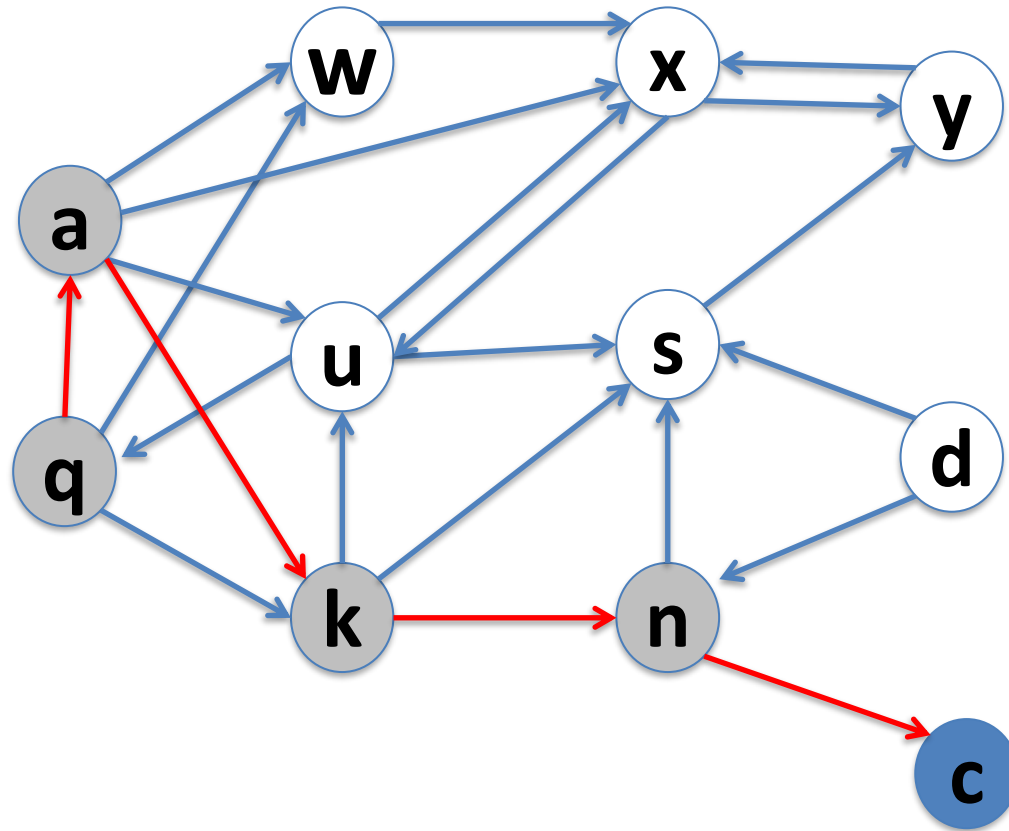
Depth-First Search



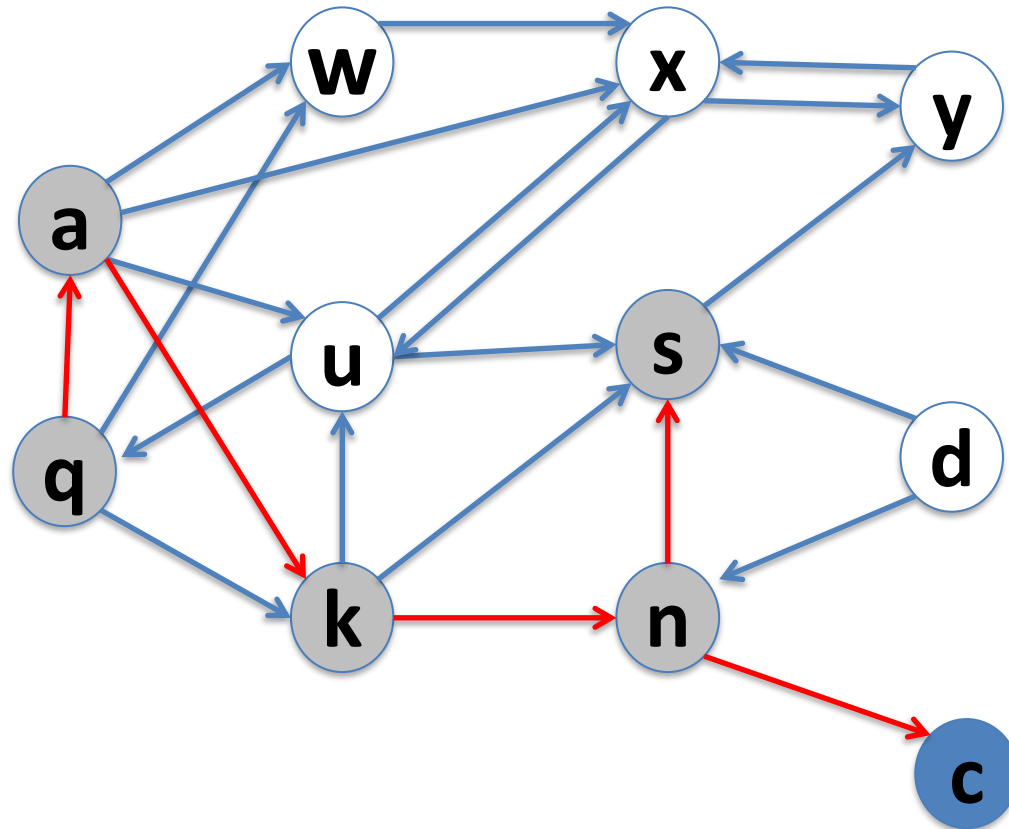
Depth-First Search



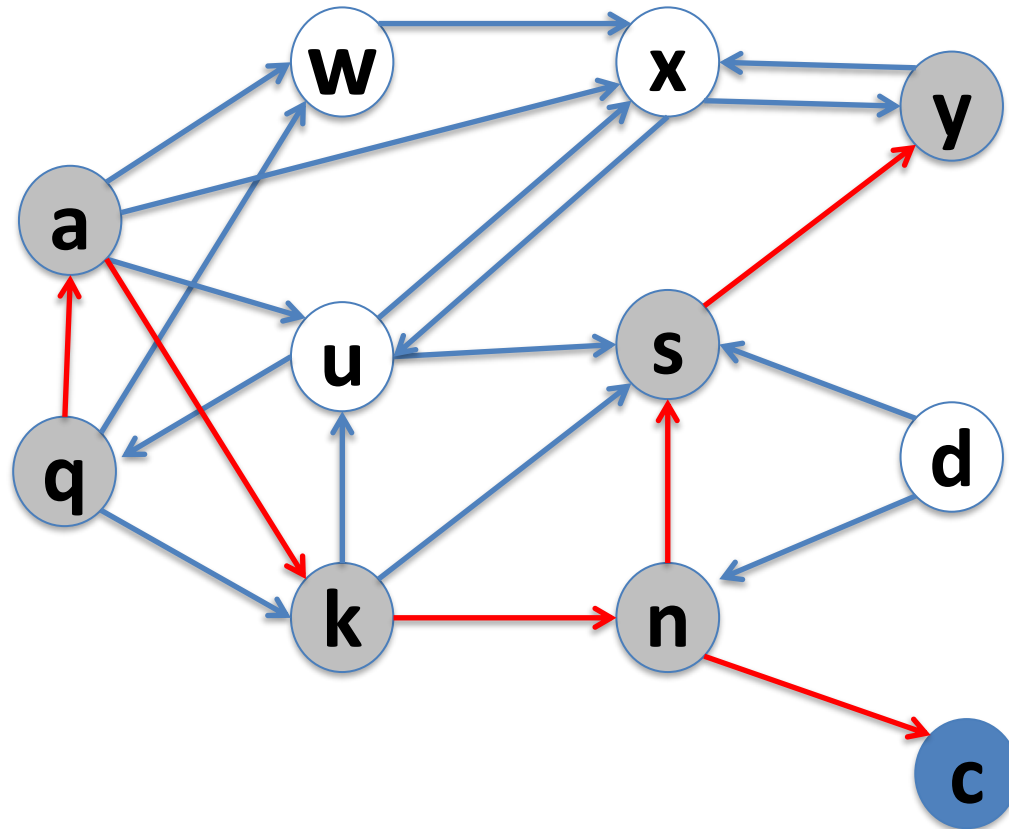
Depth-First Search



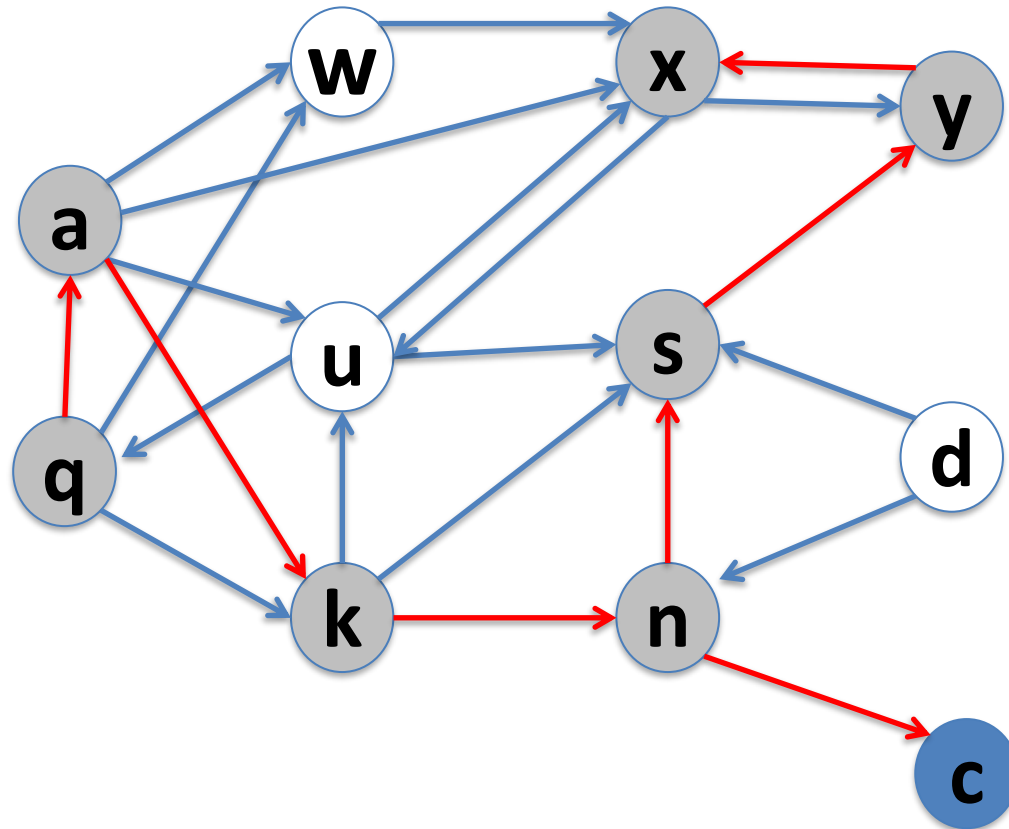
Depth-First Search



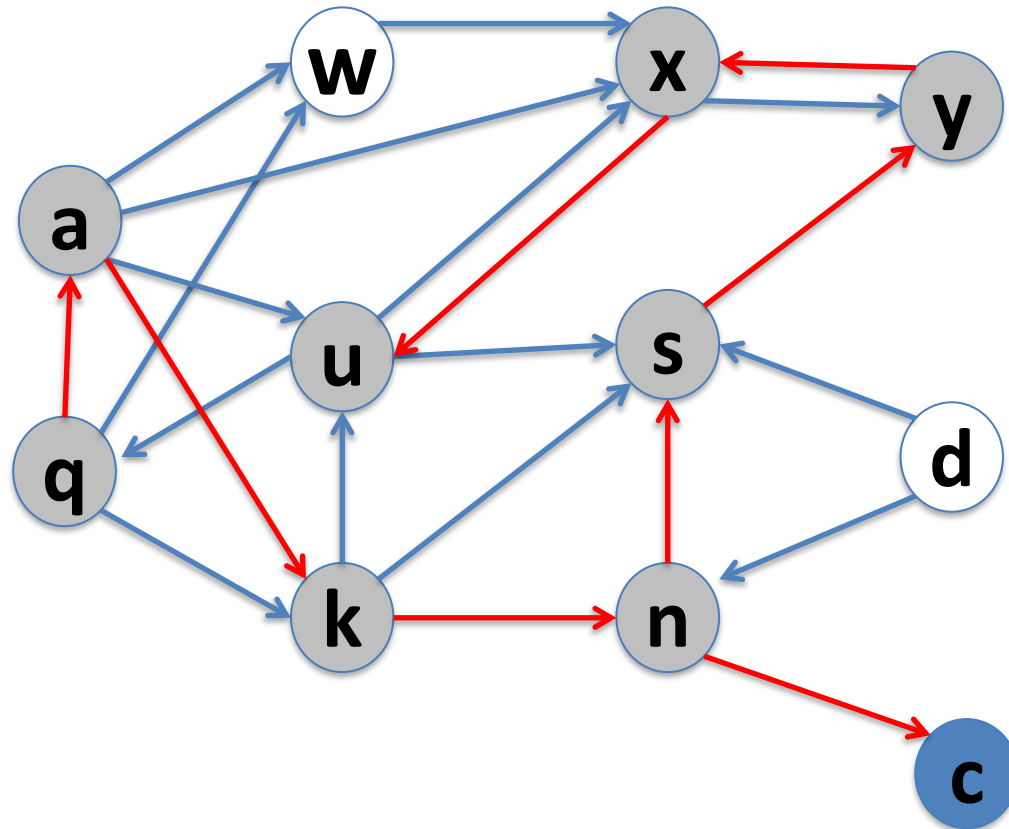
Depth-First Search



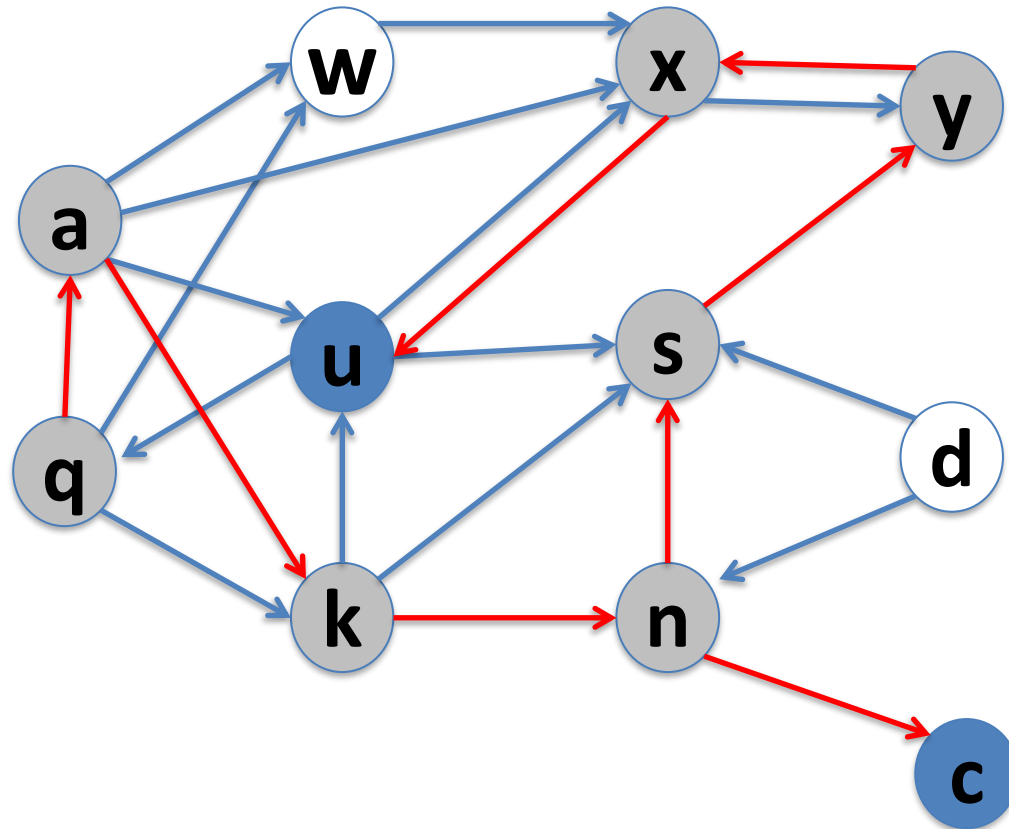
Depth-First Search



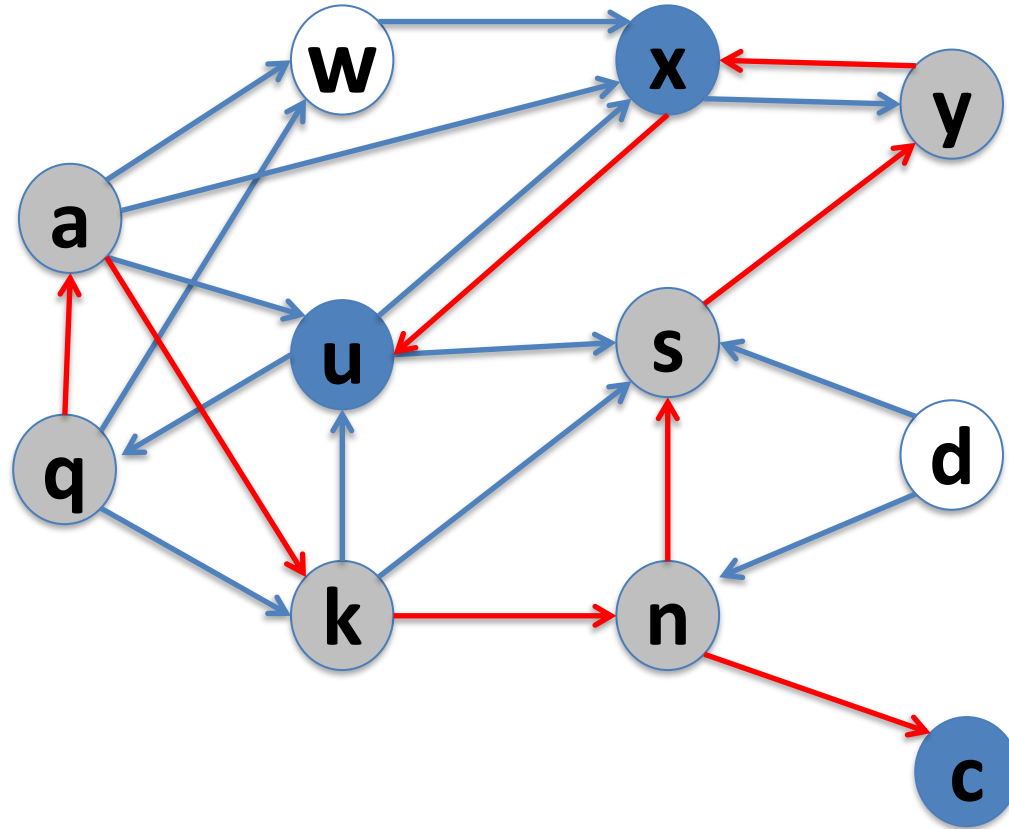
Depth-First Search



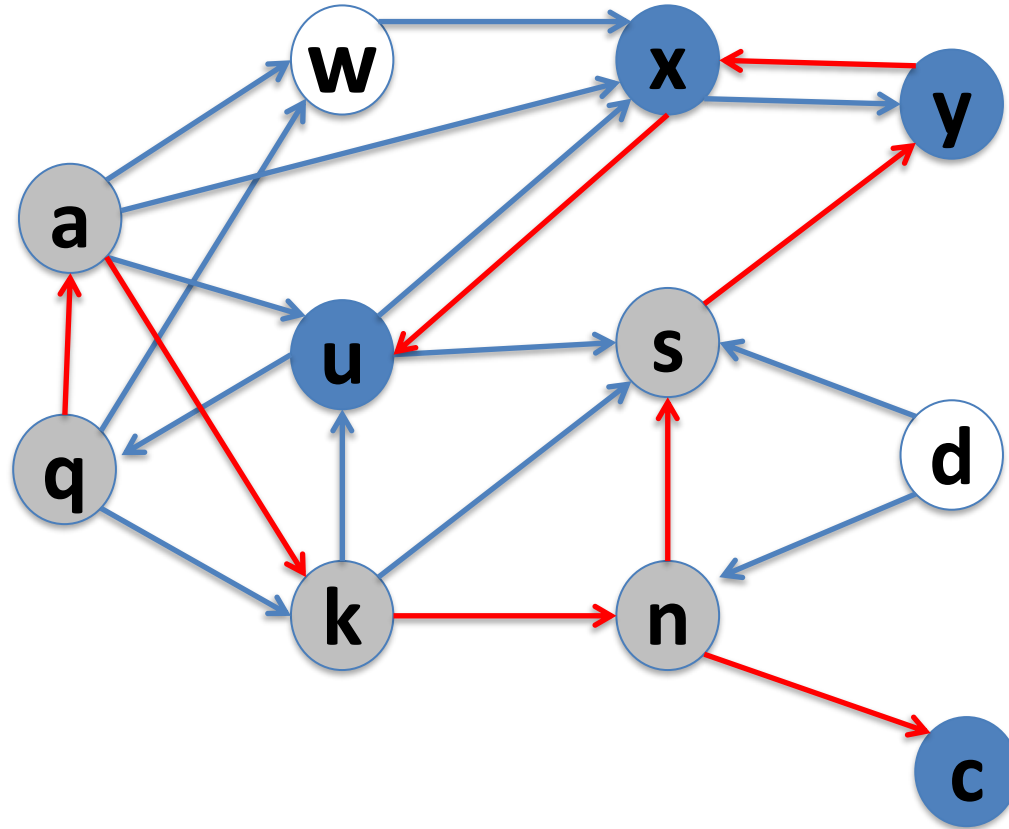
Depth-First Search



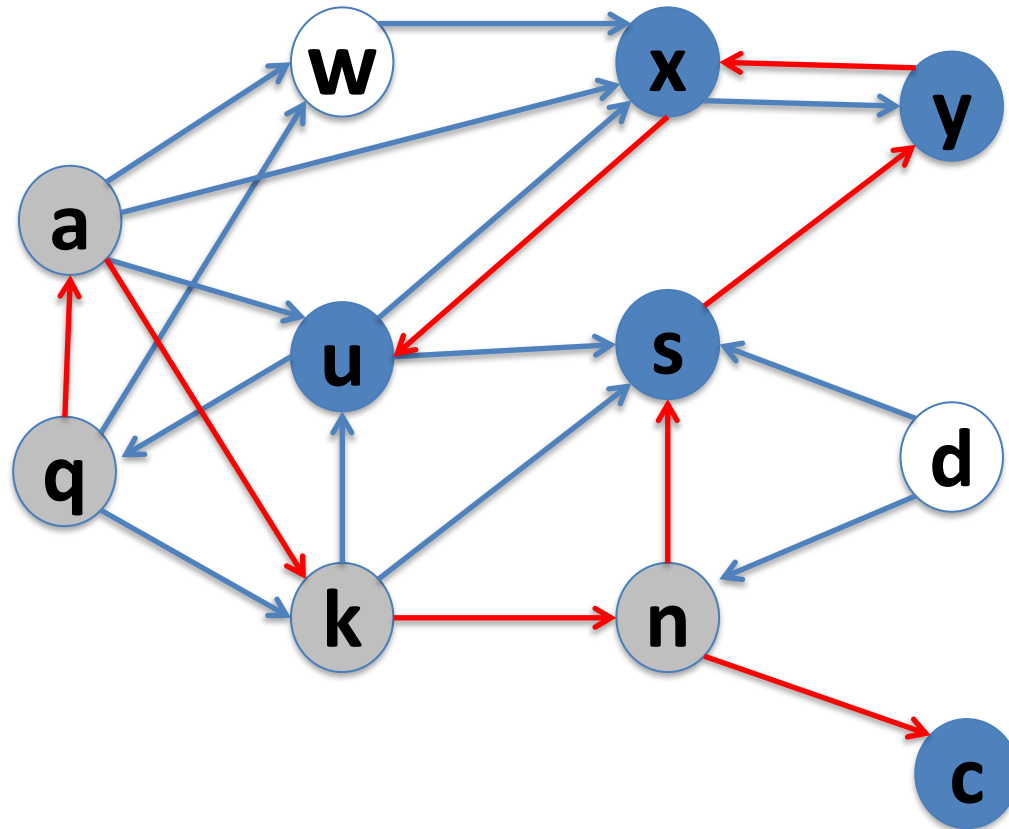
Depth-First Search



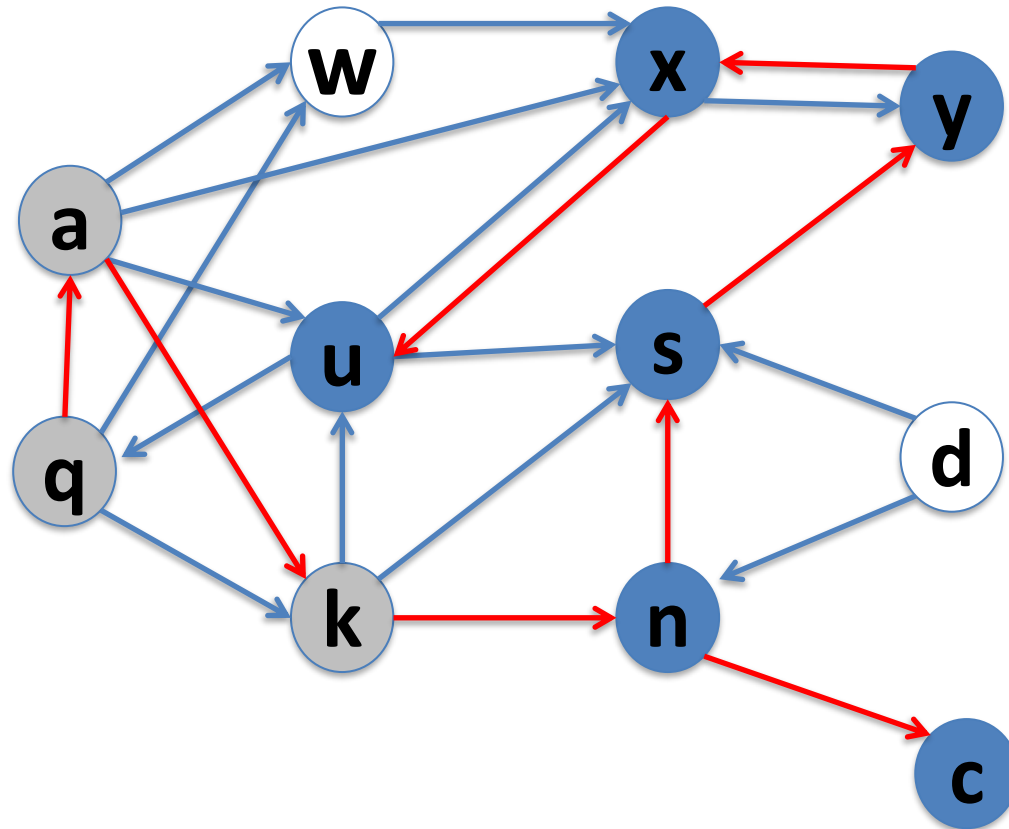
Depth-First Search



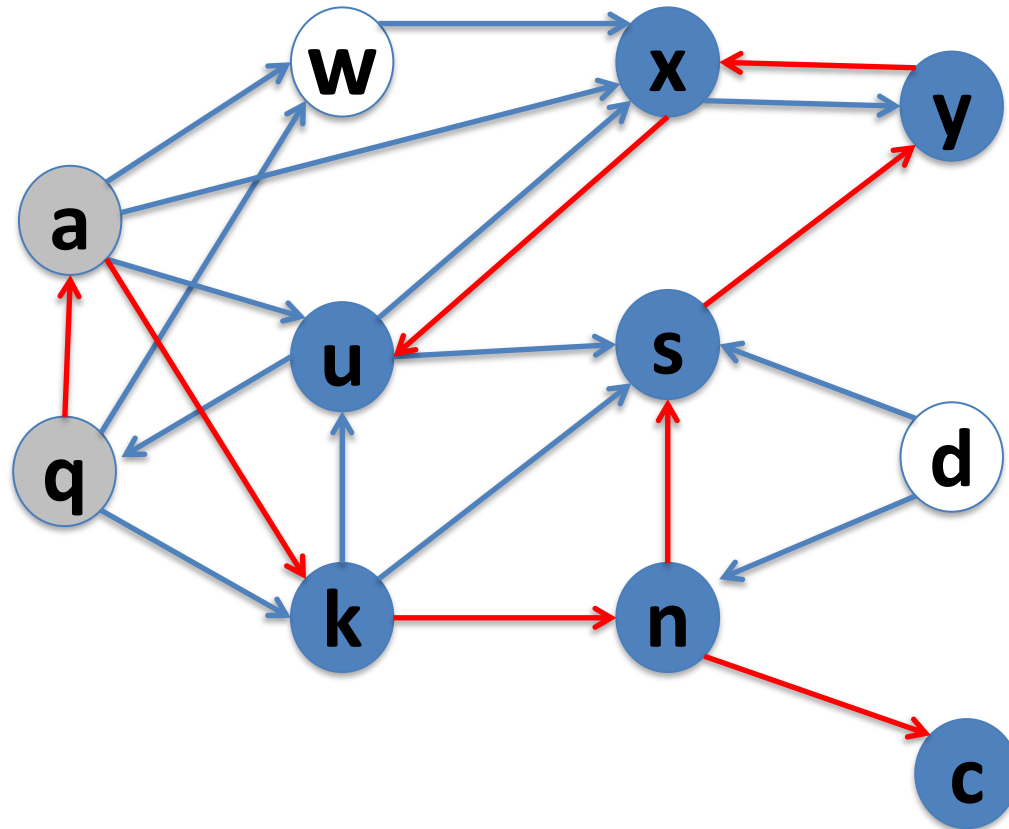
Depth-First Search



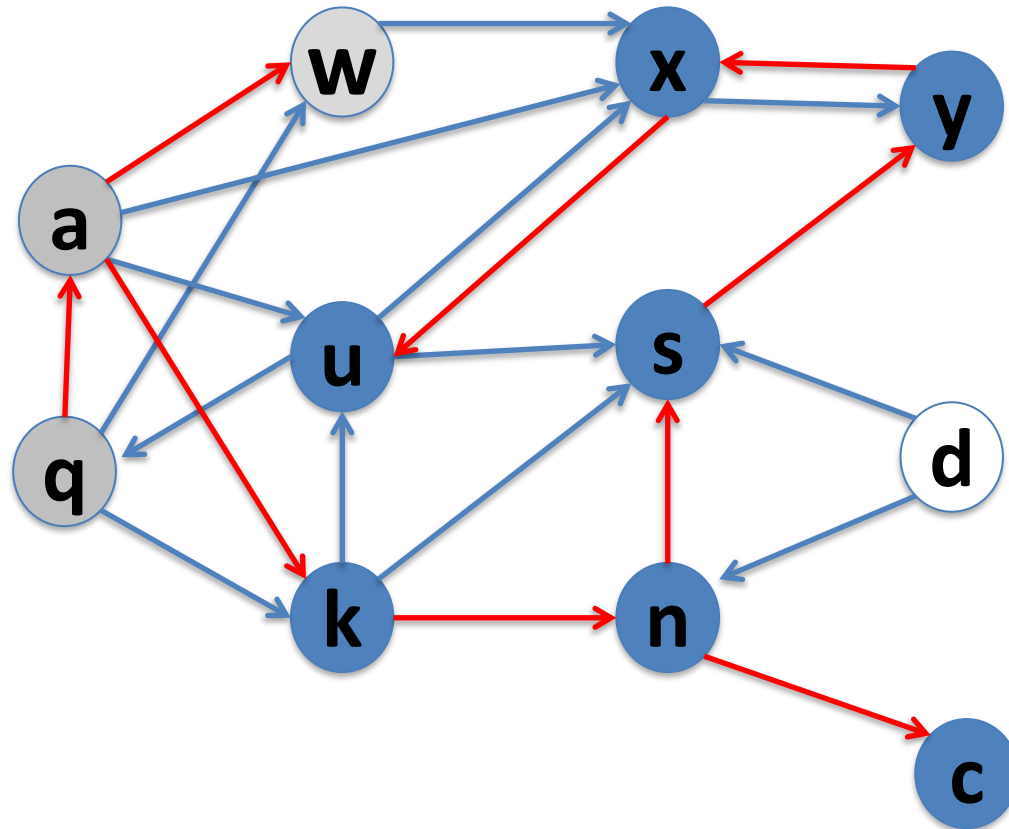
Depth-First Search



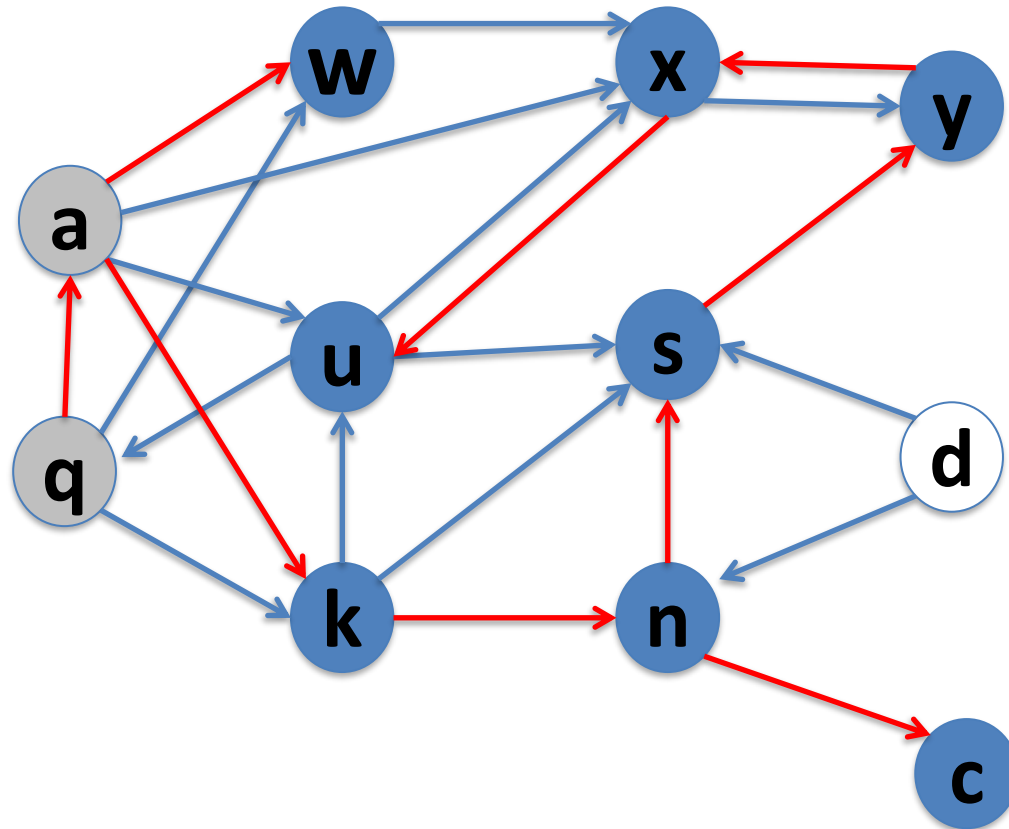
Depth-First Search



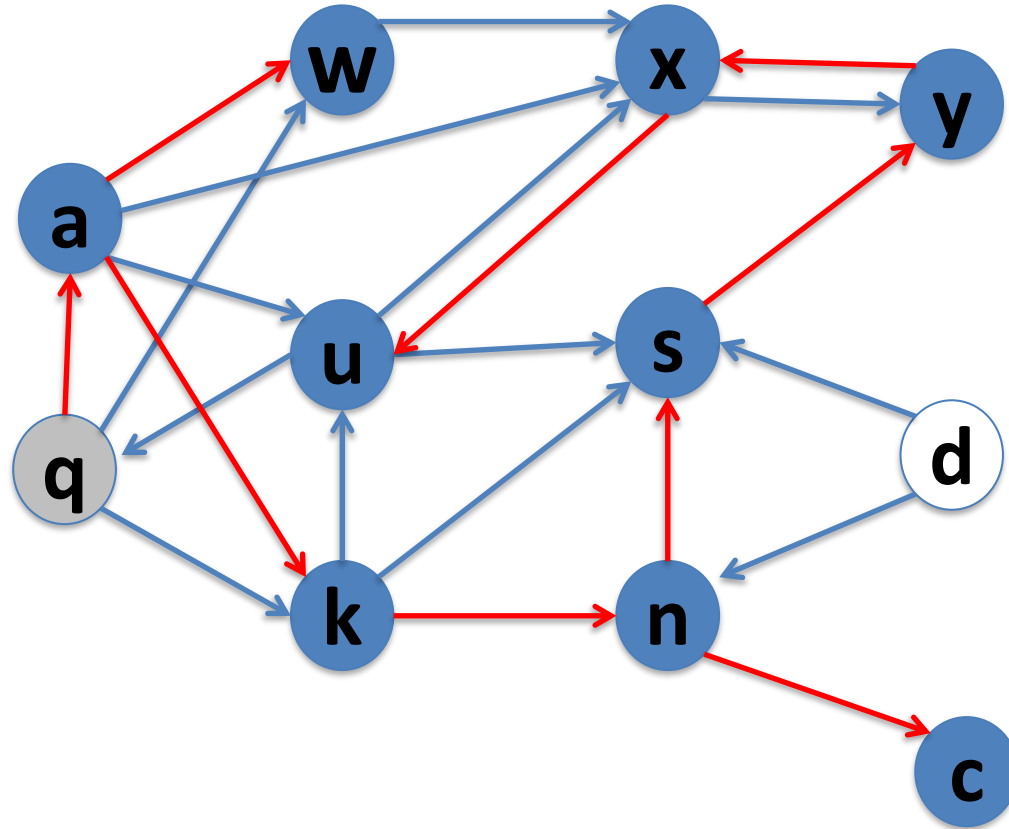
Depth-First Search



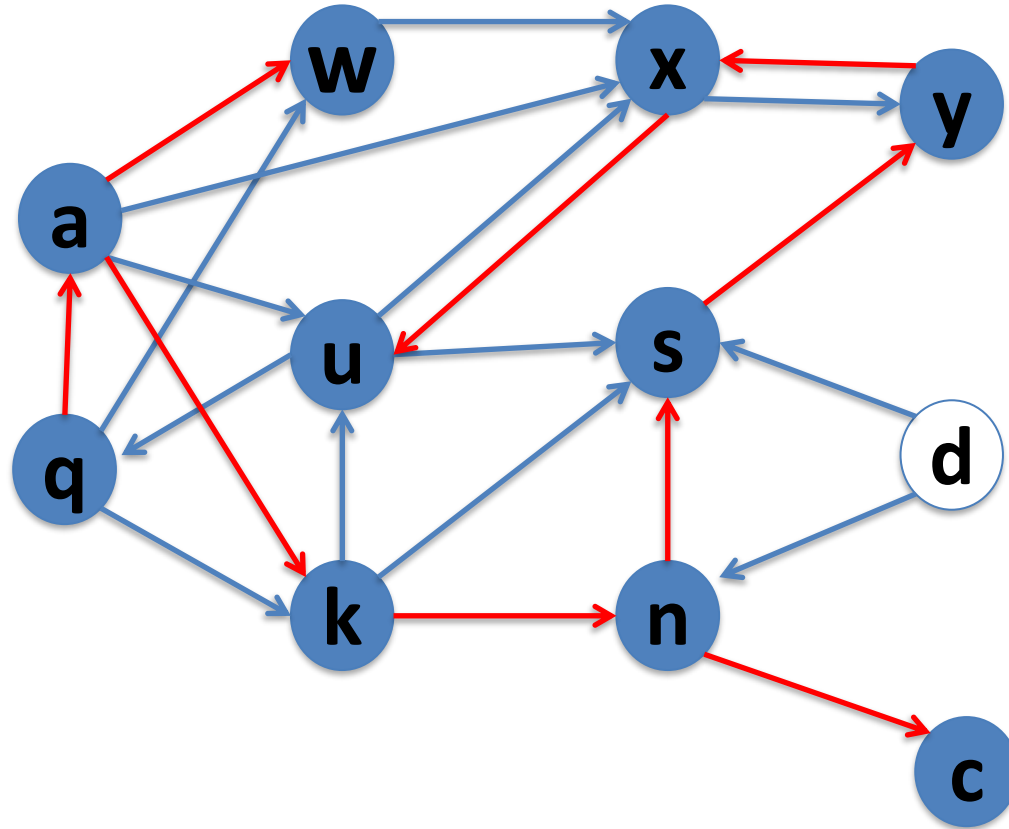
Depth-First Search



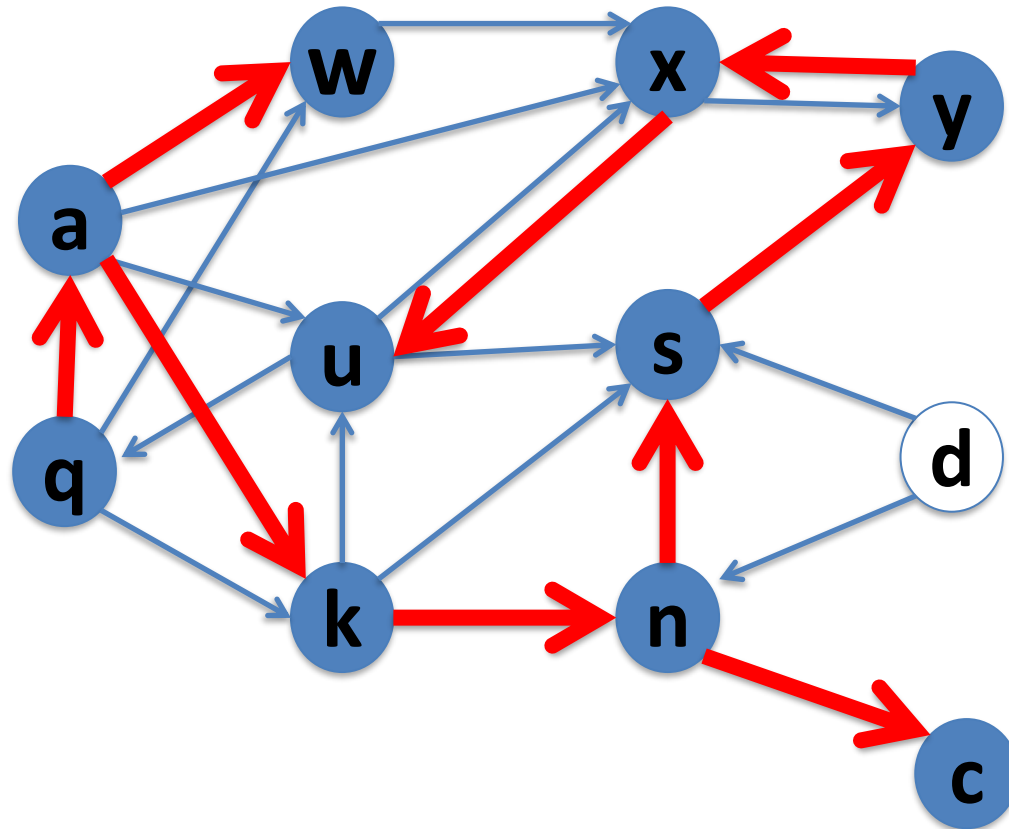
Depth-First Search



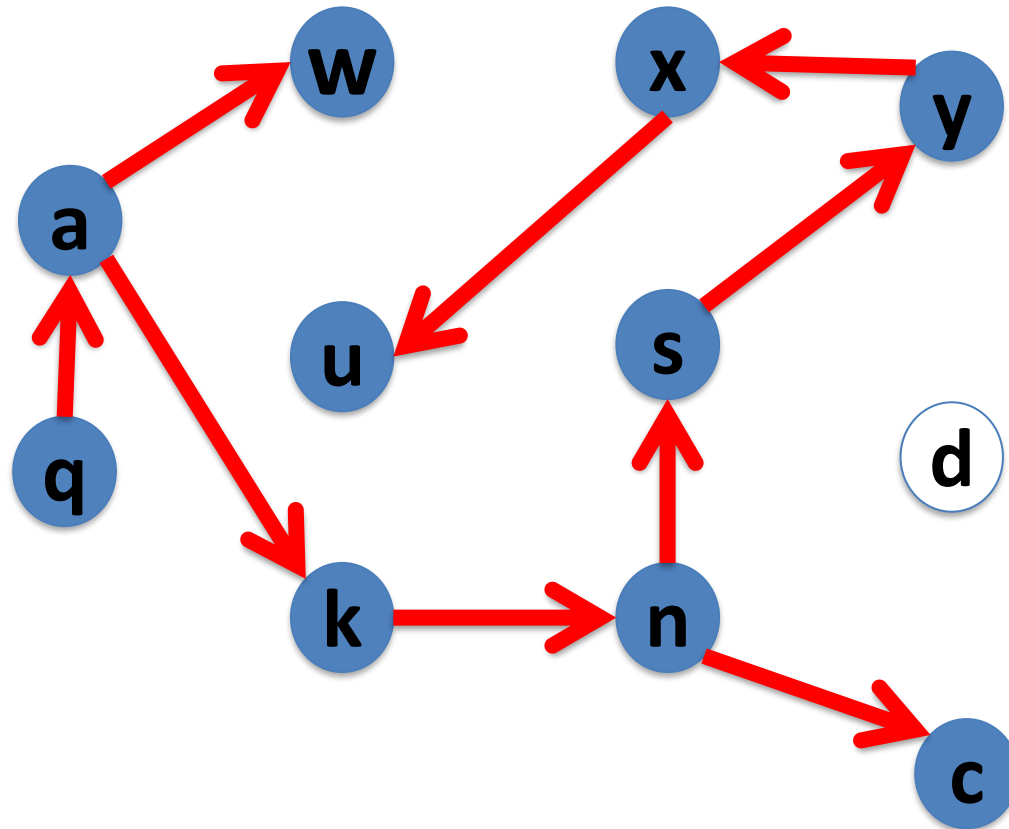
Depth-First Search



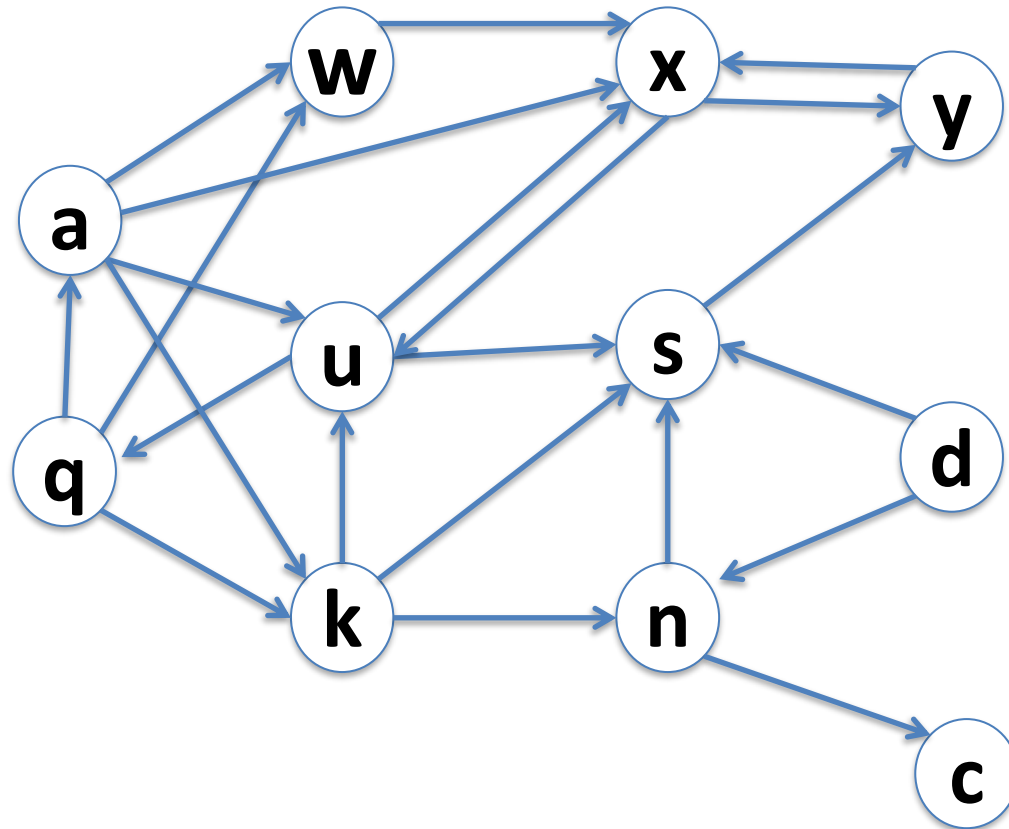
Depth-First Search



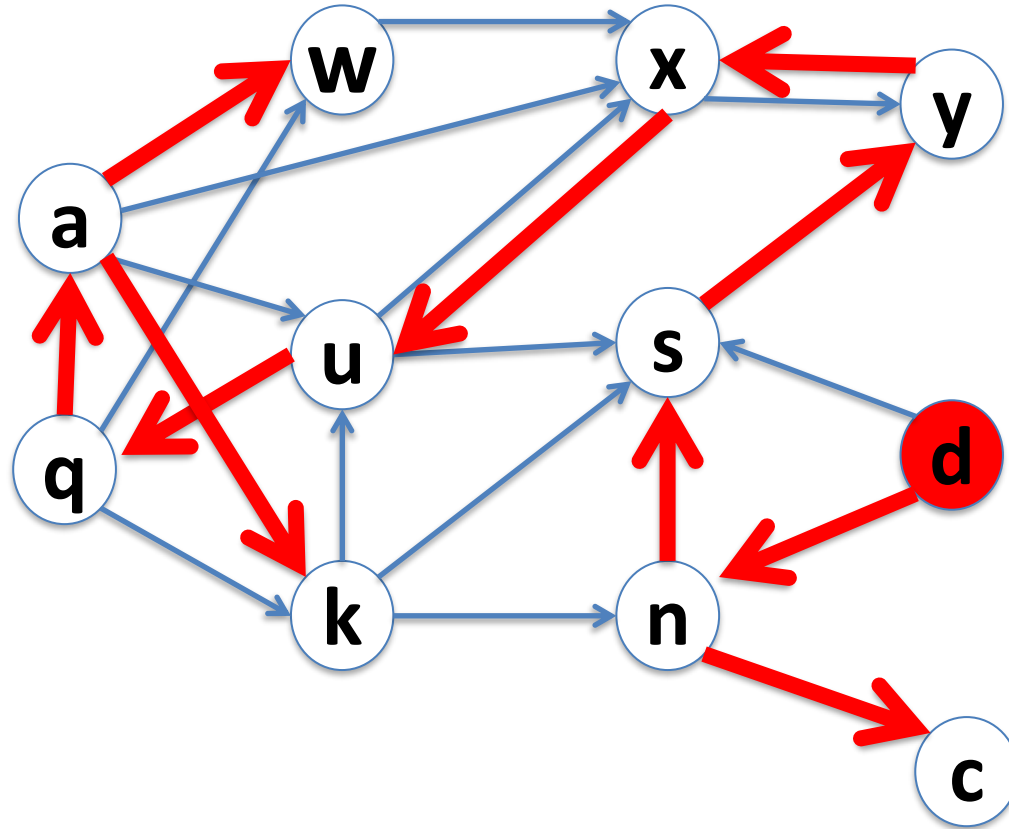
Depth-First Search



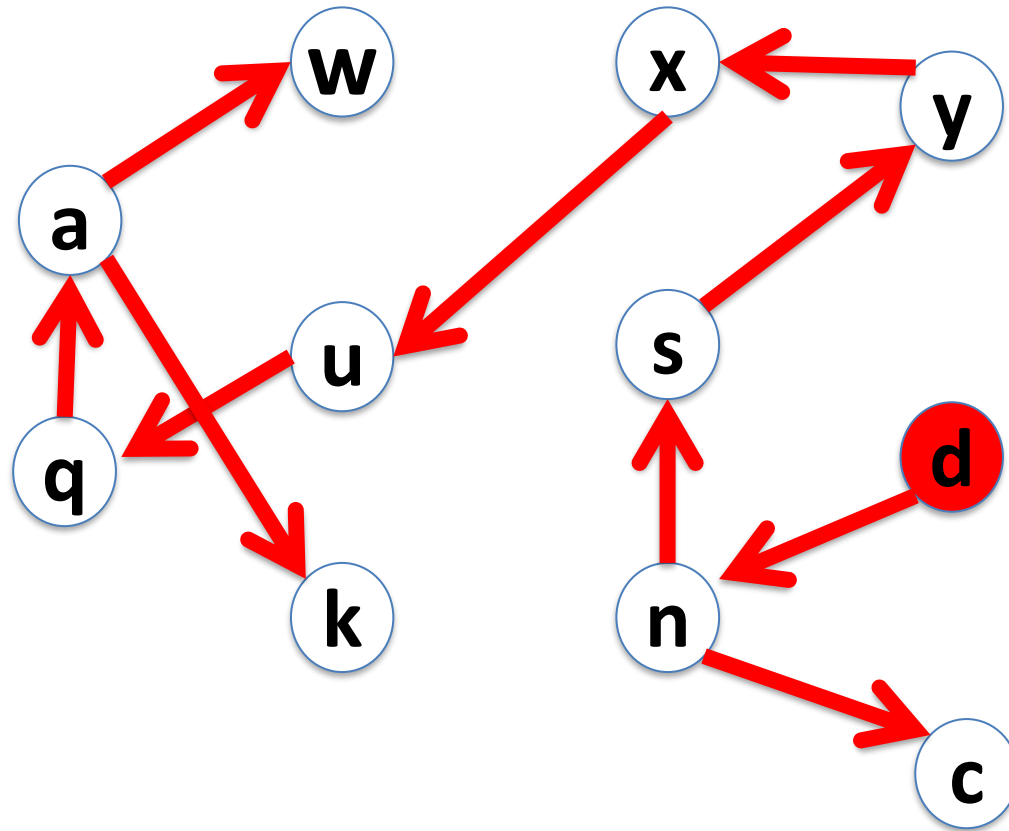
Depth-First Search



Depth-First Search



Depth-First Search

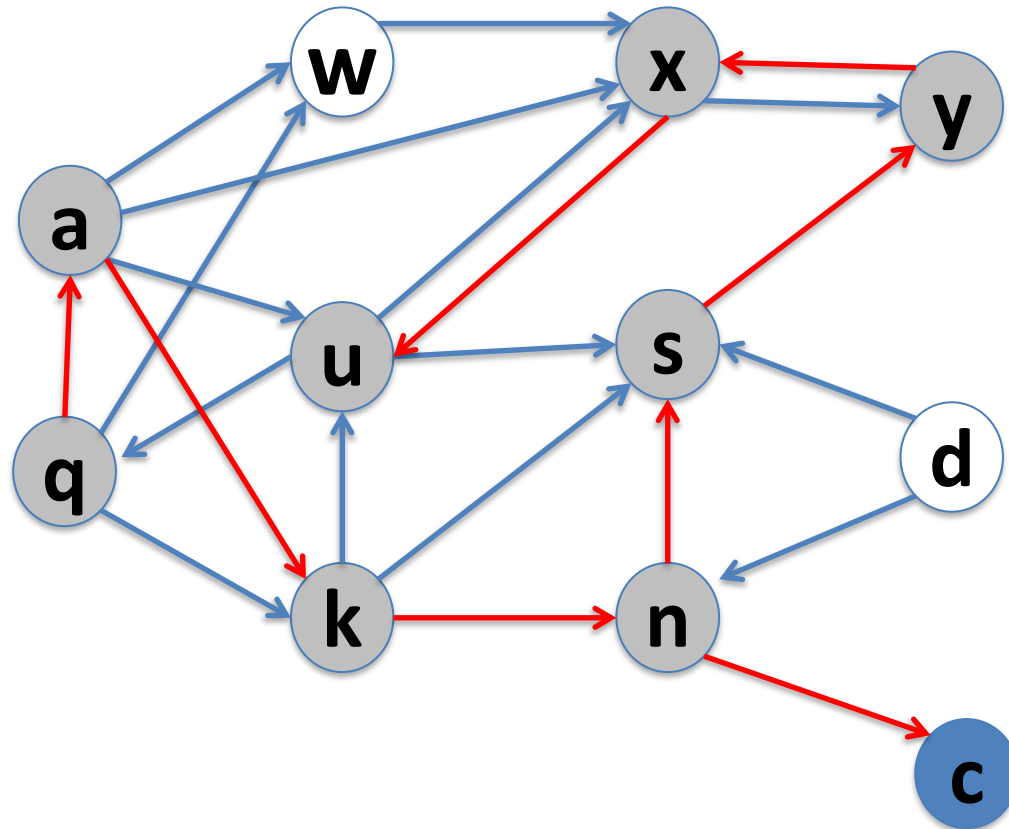


Depth-First Search

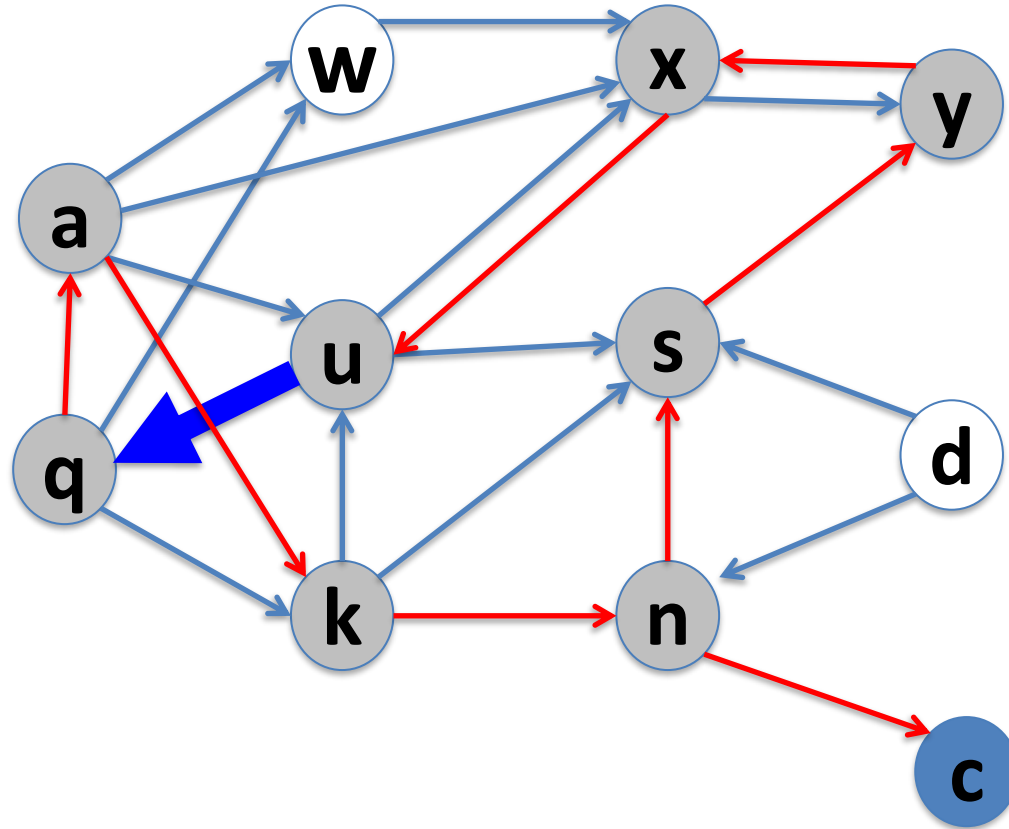
What is the runtime of DFS?

- Adjacency Matrix
- Adjacency Lists

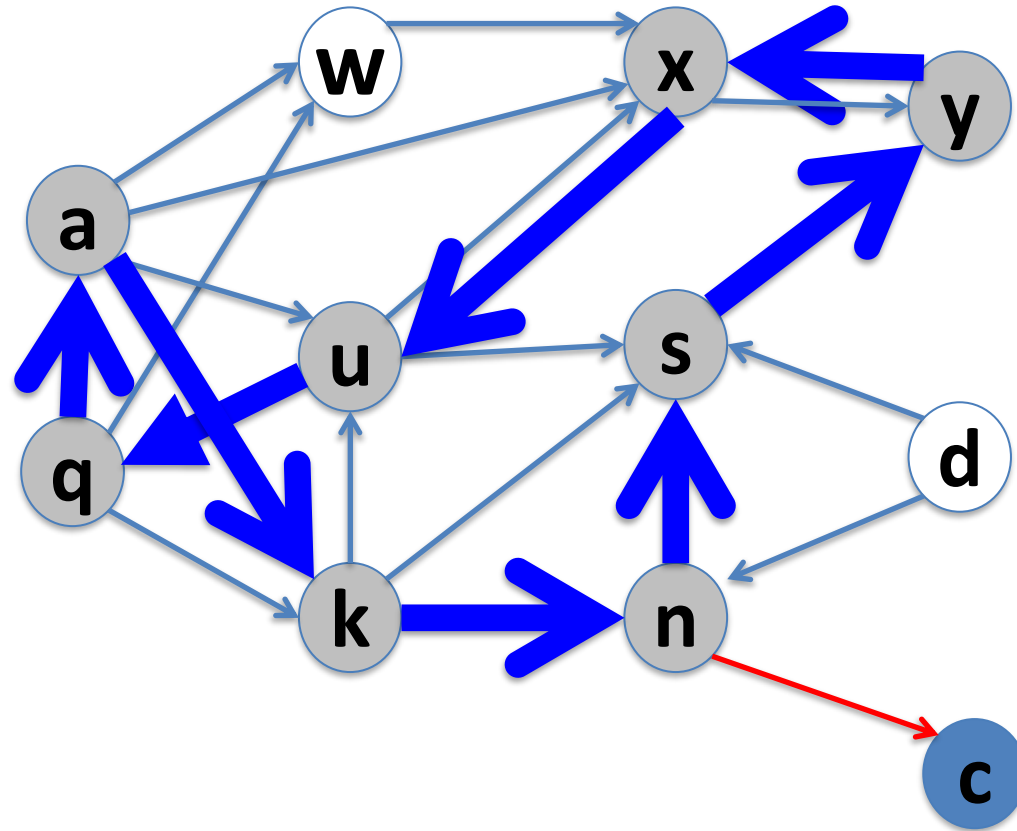
Depth-First Search



Depth-First Search



Depth-First Search



Depth-First Search

Depth-first search can be used to detect cycles in a graph.

(More about this in COMP3804)

Graph Exploration

What is the runtime of BFS and DFS?

- Adjacency Matrix

$$O(n^2)$$

- Adjacency Lists

$$O(n + m)$$